Computer Vision p1

2-08-2026

Hebah Omer Soleman

Table of Contents

Learning Outcomes

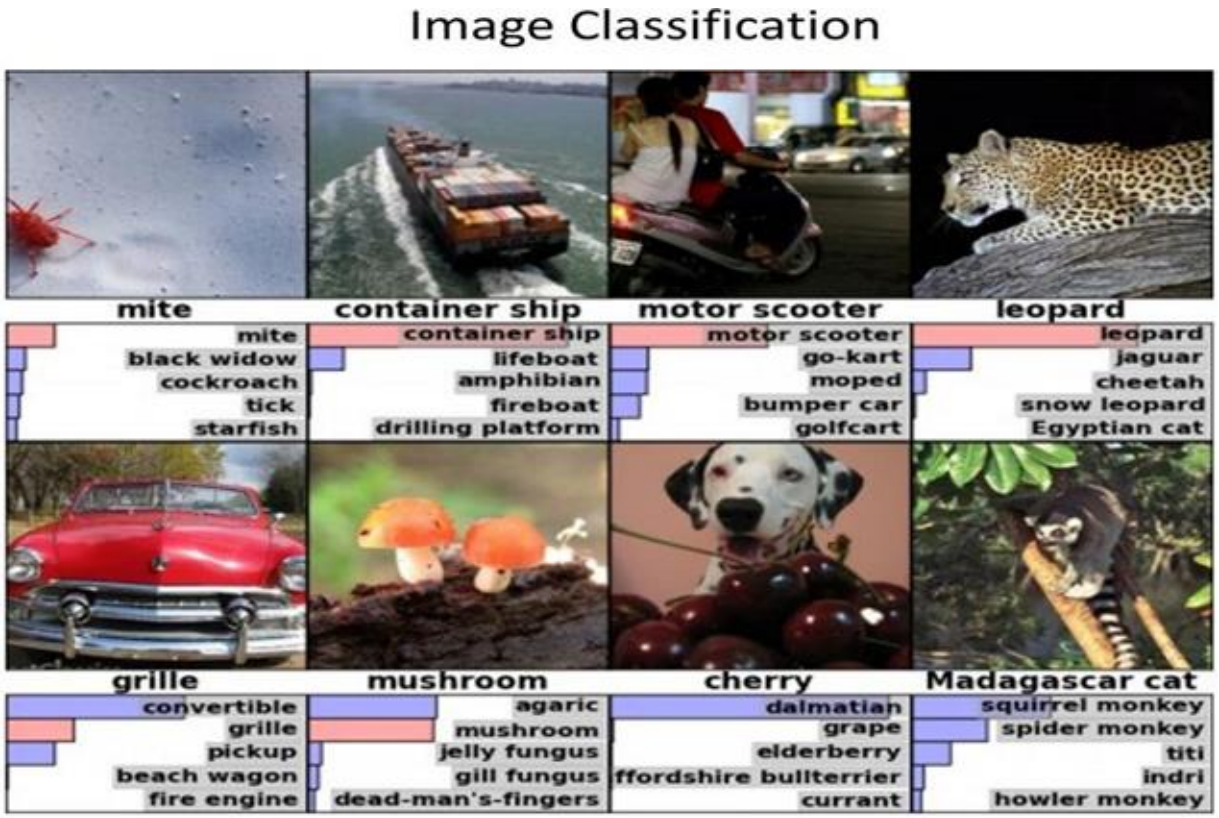
- Understand the basic concepts of Computer Vision and its real-world applications.
- Describe how images are represented and processed in a computer.
- Explain the fundamental building blocks of Convolutional Neural Networks (CNNs).
- Differentiate between popular CNN architectures (AlexNet, VGG, InceptionNet, ResNet, EfficientNet, and MobileNet) and their key innovations.

Computer Vision

Building artificial systems that process, perceive, and reason about visual data

Computer Vision is Everywhere

Some Applications

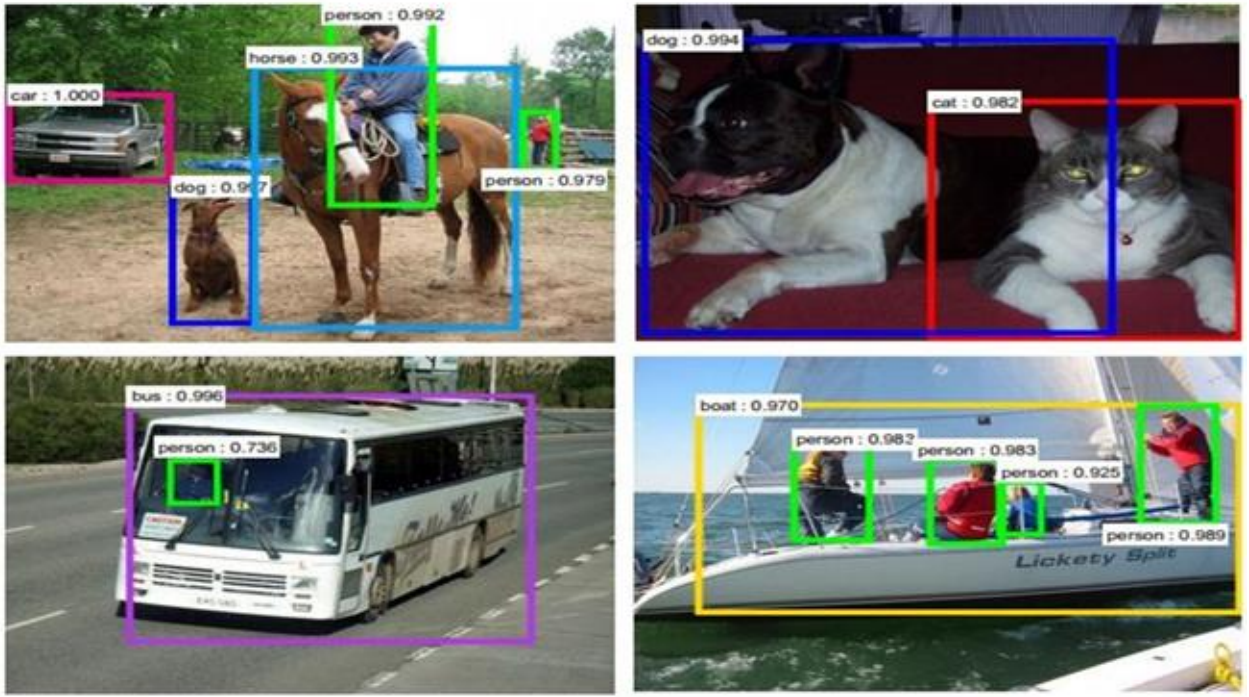


Some Applications (cont.)

Image Retrieval

Some Applications (cont.)

Object Detection



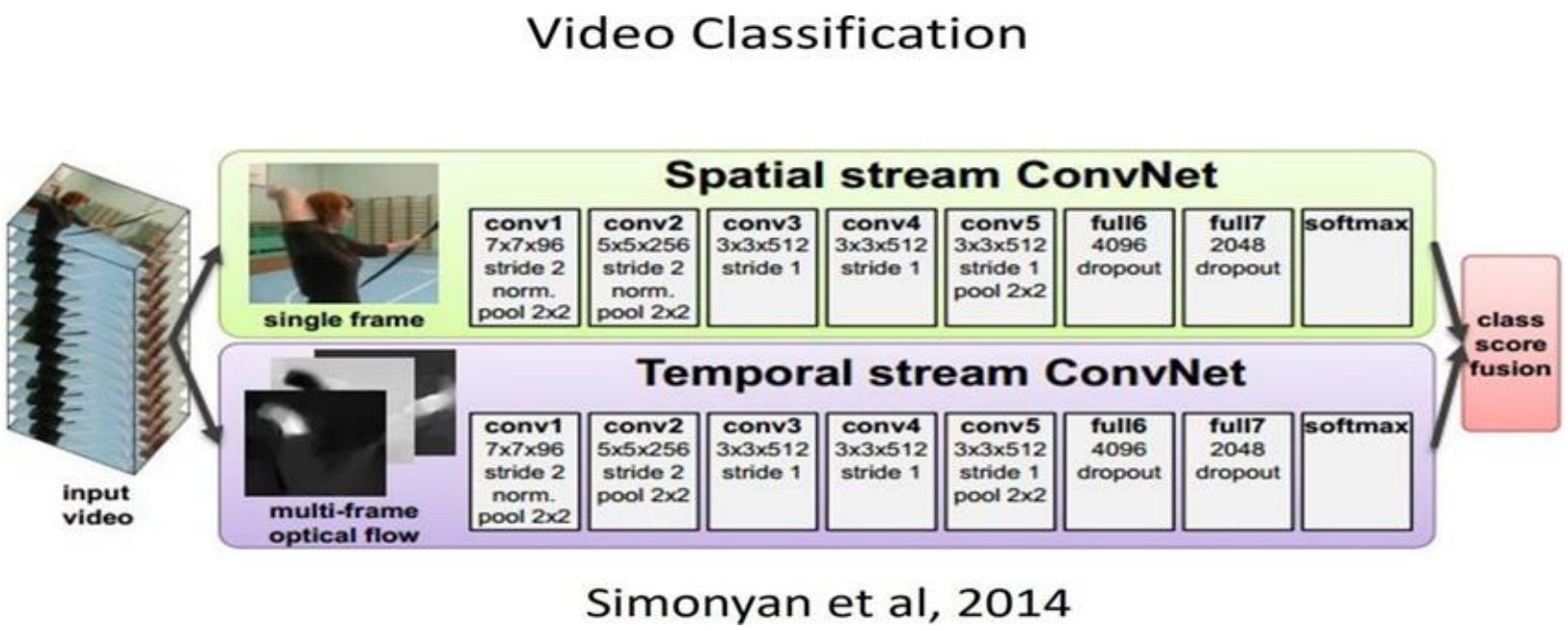
Ren, He, Girshick, and Sun, 2015

Some Applications (cont.)

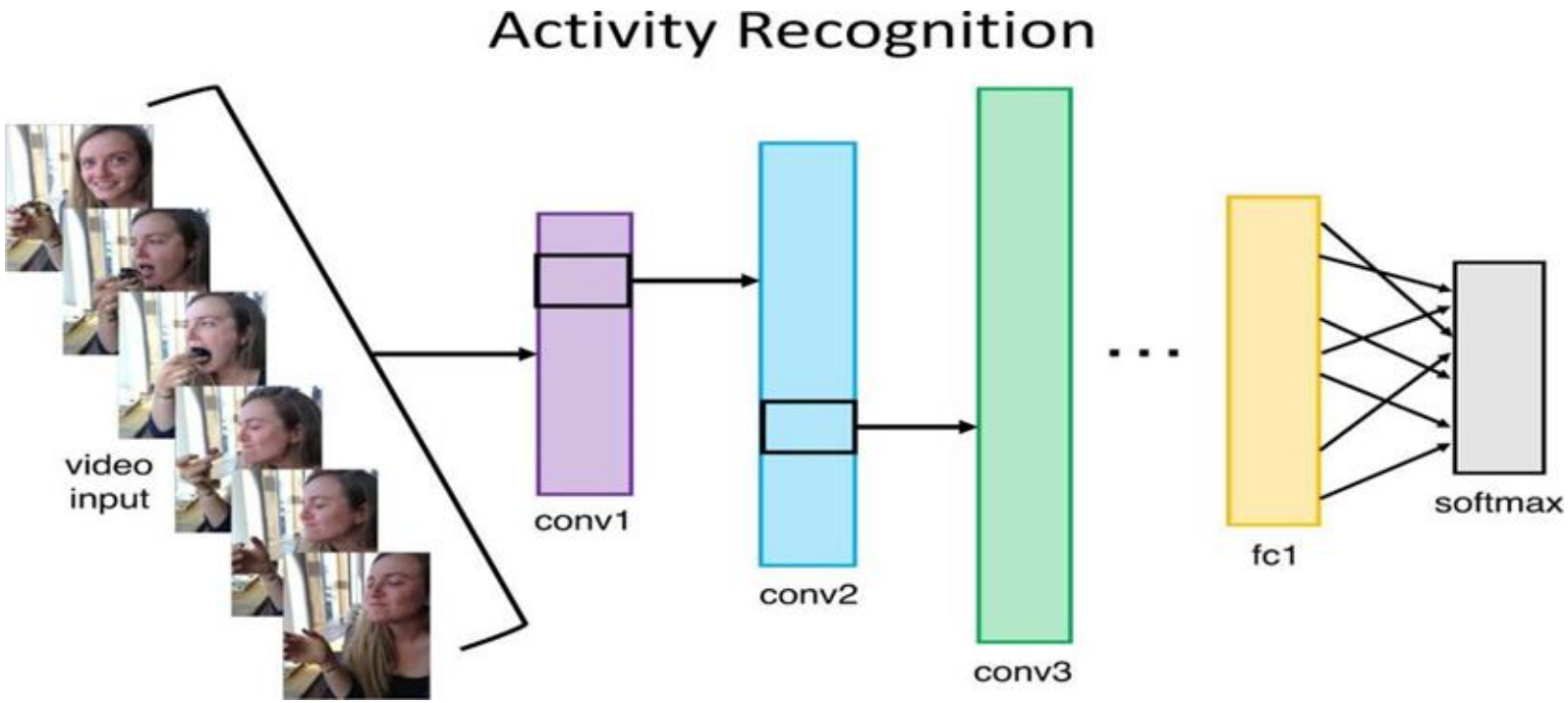
Image Segmentation

Fabaret et al, 2012

Some Applications (cont.)



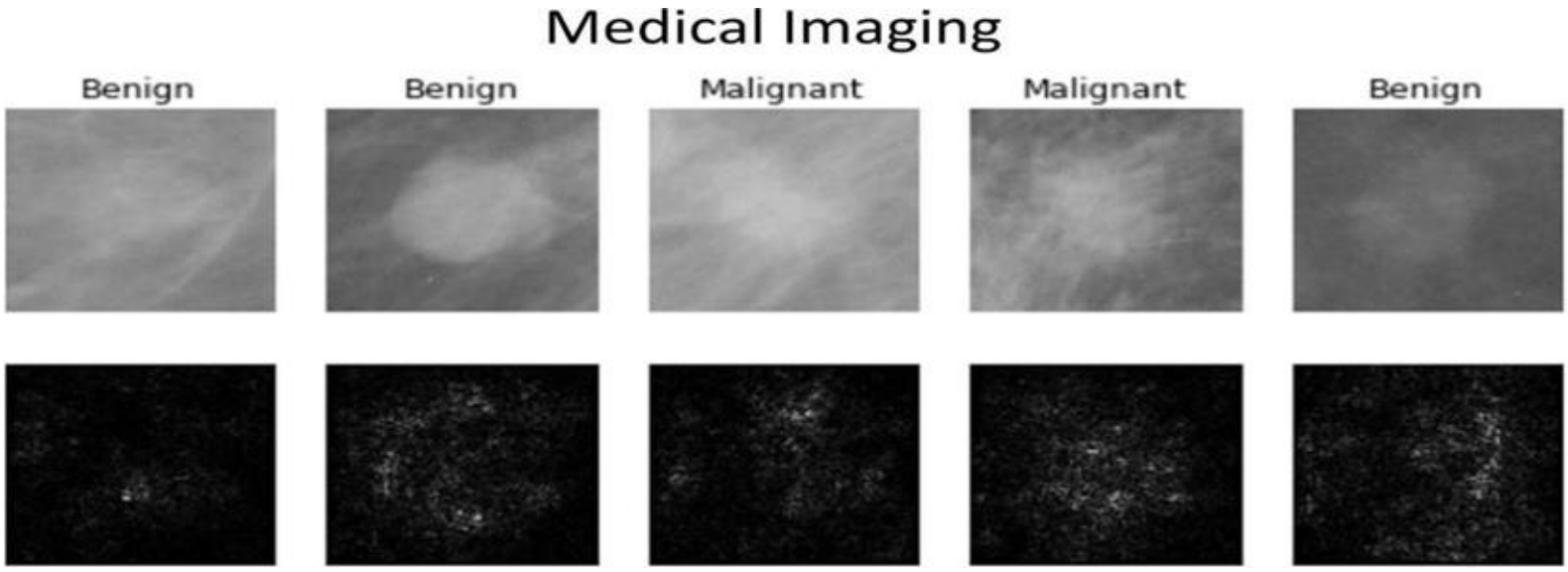
Some Applications (cont.)



Some Applications (cont.)

Pose Recognition (Toshev and Szegedy, 2014)

Some Applications (cont.)



Some Applications (cont.)

Image Captioning
Vinyals et al, 2015
Karpathy and Fei-Fei, 2015

A white teddy bear sitting in the grass

A man in a baseball uniform throwing a ball

A woman is holding a cat in her hand

A man riding a wave on top of a surfboard

A cat sitting on a suitcase on the floor

A woman standing on a beach holding a surfboard

Some Applications (cont.)

Image Generation

"Teddy bears working on new AI research underwater with 1990s technology"

DALL-E 2

Some Applications (cont.)

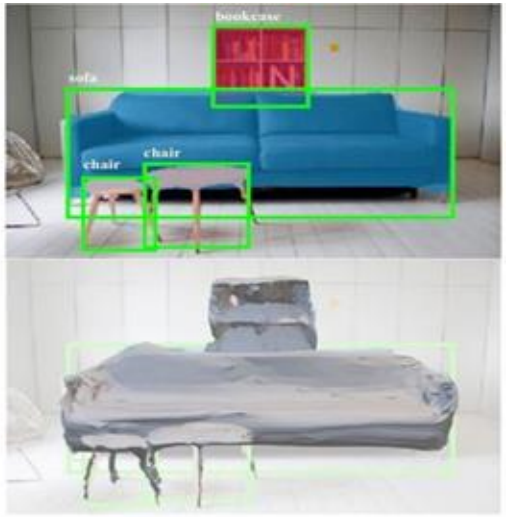
Style Transfer

Some Applications (cont.)

3D Vision

Choy et al., 3D-R2N2: Recurrent Reconstruction Neural Network (2016)

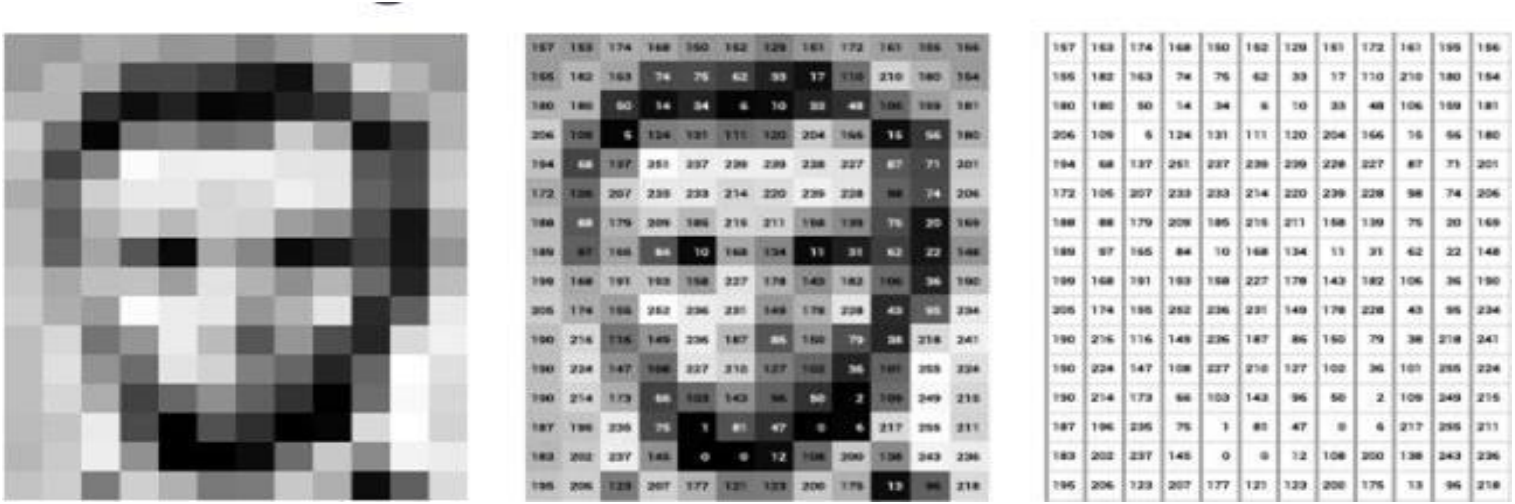
Zhou et al., 3D Shape Generation and Completion through Point-Voxel Diffusion (2021)



Gkioxari et al., "Mesh R-CNN", ICCV 2019

How to represent an image?

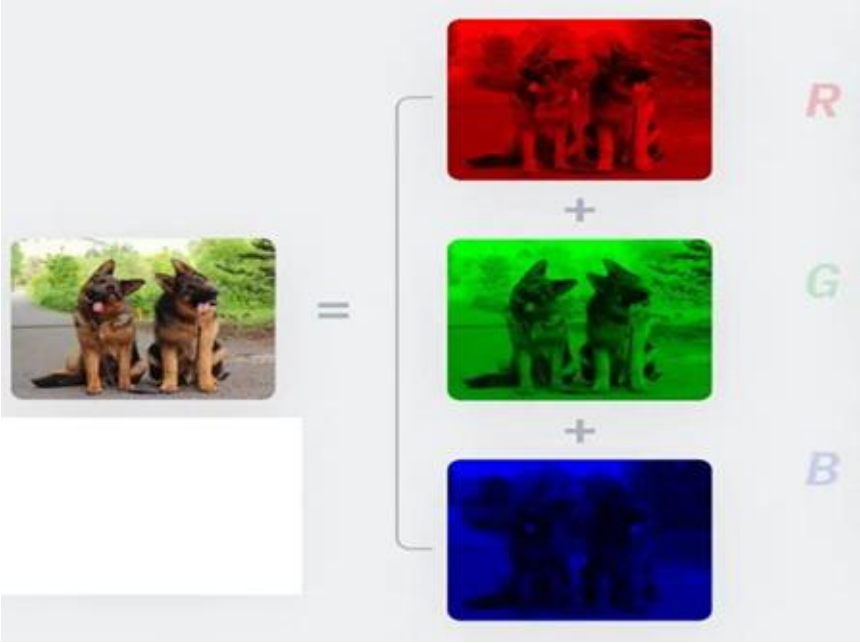
- Images are represented as Matrices with elements in[255 ,0]
- Grayscale images have one channel.



[0]https://www.v7labs.com/blog/image-recognition-guide

How to represent an image?

- RGB images have 3 channels.



How can we build vision models?

- So far, we've worked with tabular data, where each sample consists of structured features. We processed this data using Fully Connected Neural Networks.

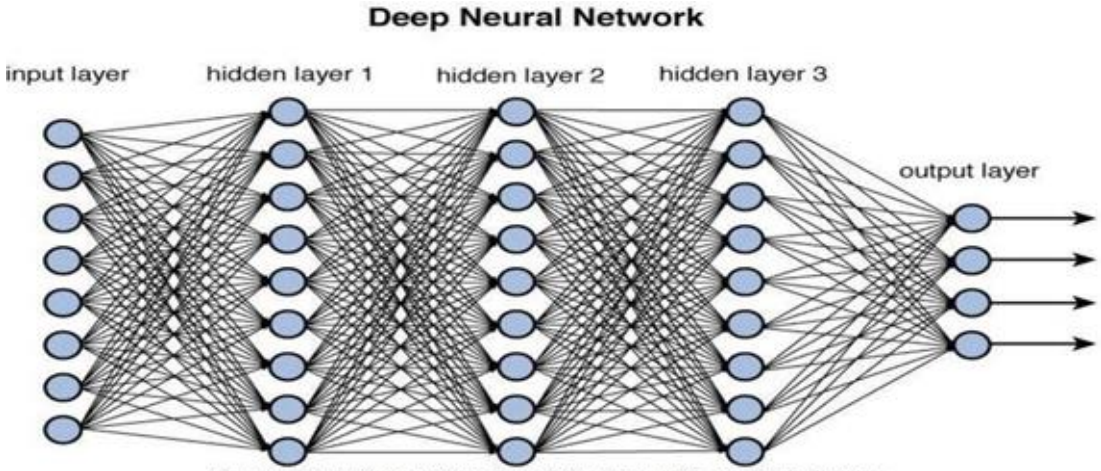


Figure 12.2 Deep network architecture with multiple layers.

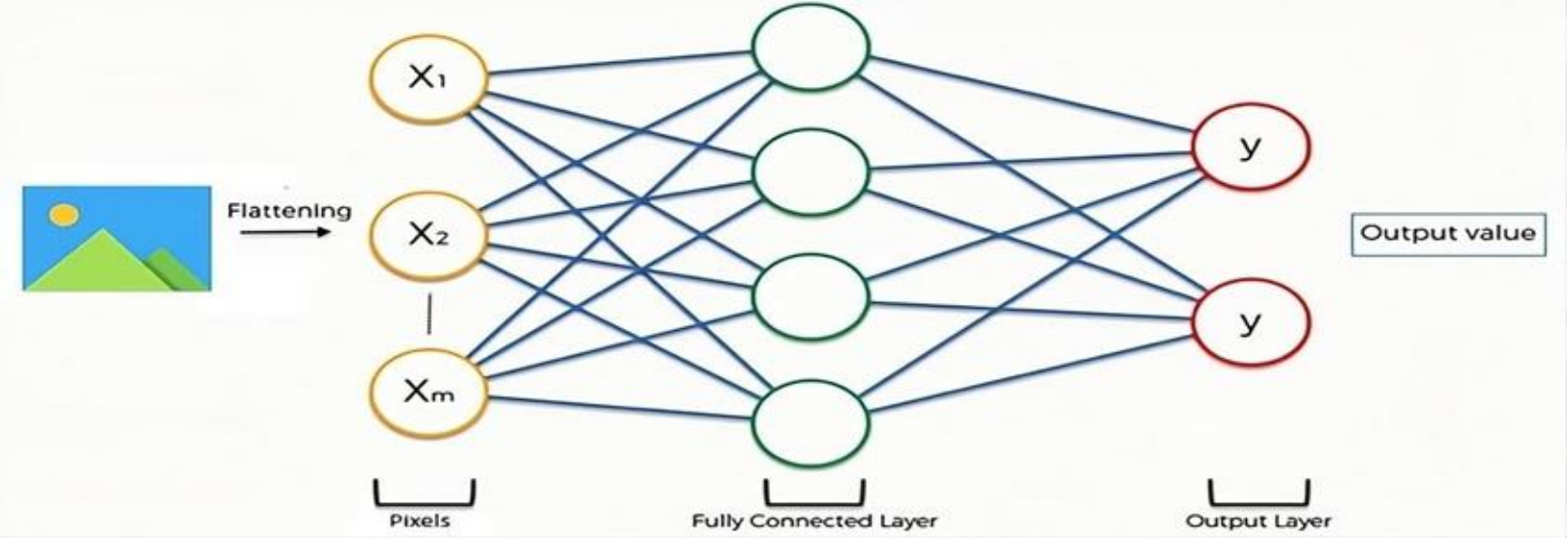
- But can we apply the same approach to images?

How can we build vision models?

- Let’s try.

How can we build vision models?

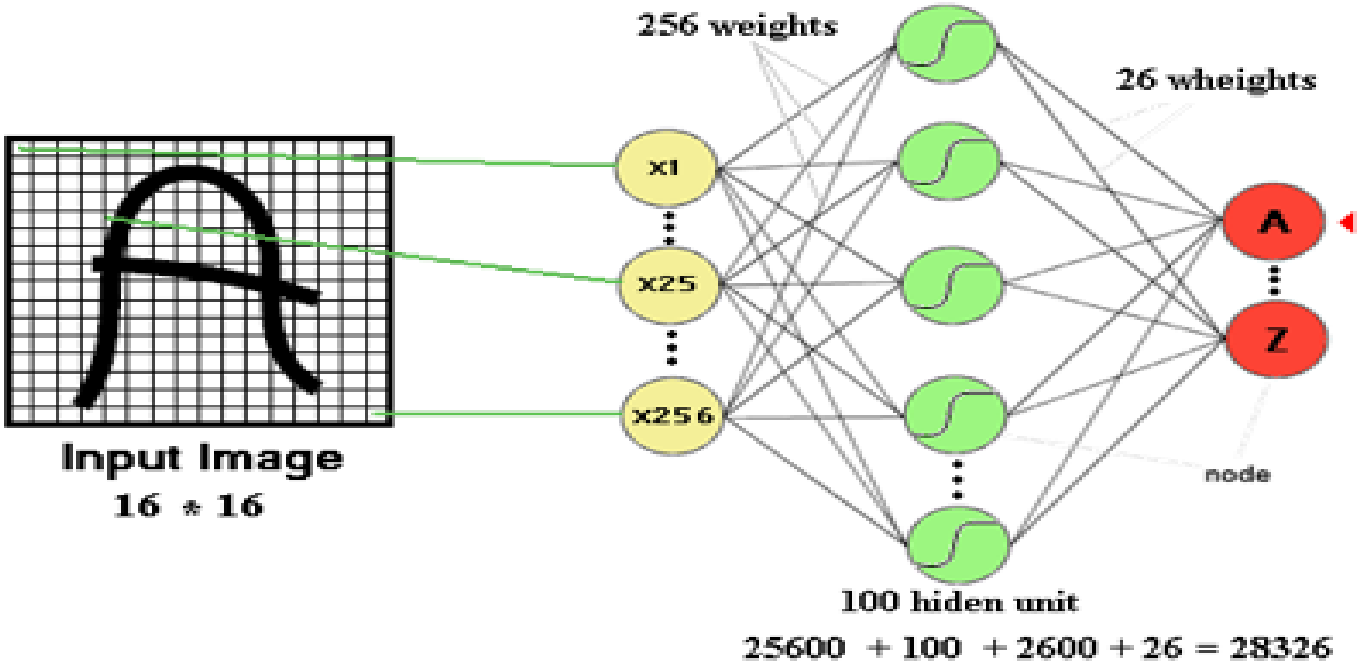
- Let's consider each pixel as a feature.



- This should work! But...

Drawbacks of Fully-Connected Neural Networks

- The number of trainable parameters becomes extremely large



Drawbacks of Fully-Connected Neural Networks (cont.)

- Little or no invariance to shifting, scaling, and other forms of distortion
- In other words, the Fully connected NN cannot recognize that the two images (original and shifted) represent the same object (letter "A.("

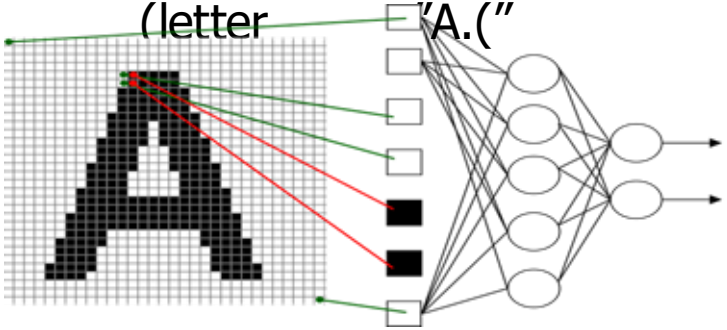


Figure " lanigirO :2A "character.

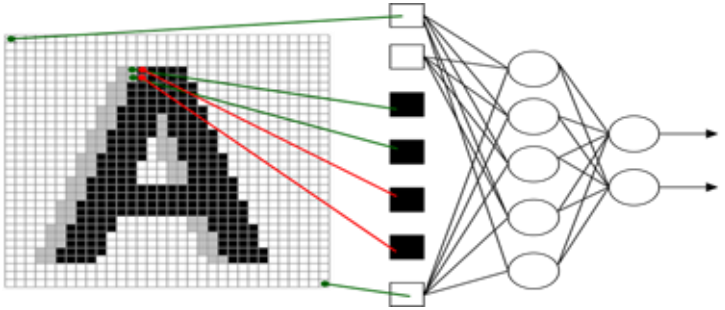


Figure " detfihS :3A "character.

Drawbacks of Fully-Connected Neural Networks (cont.)

- The topology of the input data is completely ignored (it treats pixels as independent features rather than structured patterns.)
- For a 32 ×32 image, we have
  - Black and white patterns: $2^{1024} =^{32} *2^{32}$
  - Grayscale patterns: $256^{1024} =^{32} *256^{32}$

What do we need?

- We need a model that can:
  - **Find Patterns in Images** dna segde ekil serutaef llams ezingoceR : shapes in different parts of the image.
  - **Work Efficiently** gnisucof yb yletarapes lexip yreve ta gnikool diovA : on groups of pixels together.
  - **Handle Changes** ,sevom ti fi neve tcejbo emas eht ezingocer llitS : rotates, or looks slightly different.

What do we need?

- We need a model that can:
  - **Find Patterns in Images** dna segde ekil serutaef llams ezingoceR : shapes in different parts of the image.
  - **Work Efficiently** gnisucof yb yletarapes lexip yreve ta gnikool diovA : on groups of pixels together.
  - **Handle Changes** ,sevom ti fi neve tcejbo emas eht ezingocer llitS : rotates, or looks slightly different.
  - We need **Convolutional Neural Networks (CNNs)!**

Convolutional Neural Networks (CNNs)

- CNNs are neural networks designed for image processing and consist of two main parts:
  - **Feature Extractor** ,segde sa hcus (serutaef) snrettap snrael yllacitamotuA : .egami eht morf sepahs dna ,serutxet
  - **Classifier** a ot dessap dna rotcev a otni denettalf era serutaef detcartxe ehT : .noitacifissalc rof (erofeb desu ew seno eht ekil) krowten laruen dradnats

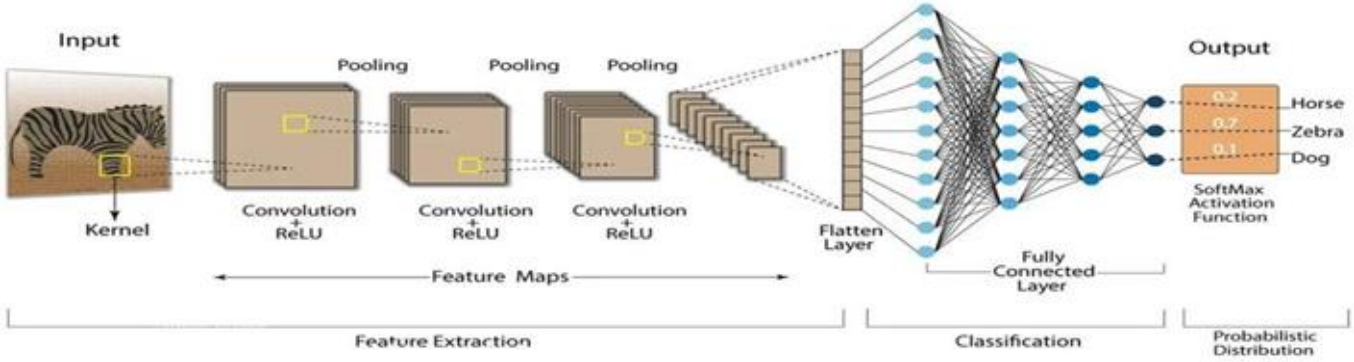


Figure krowteN larueN lanoitulovnoC :4

Convolutional Neural Networks (CNNs)

- The feature extractor consists of three essential components:
  - **Convolution Layers** ,serutxet ,segde sa hcus snrettap lacol tceteD :
    and shapes by applying filters to the image.
  - **Activation Function (ReLU)**.ytiraenil-non sddA :
  - **Pooling Layers**.serutaef detcartxe fo ezis laitaps eht ecudeR :
- Let's start with the convolution layer.

How Convolution Works?

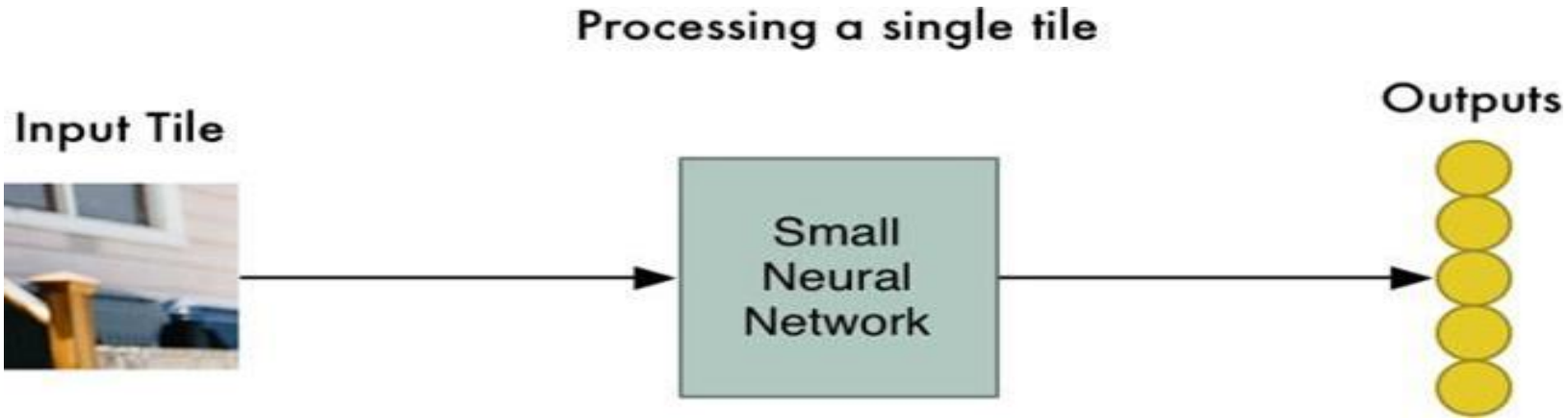
- Let’s consider this image.

How Convolution Works? (cont.)

.1The image is divided into small overlapping tiles (regions).

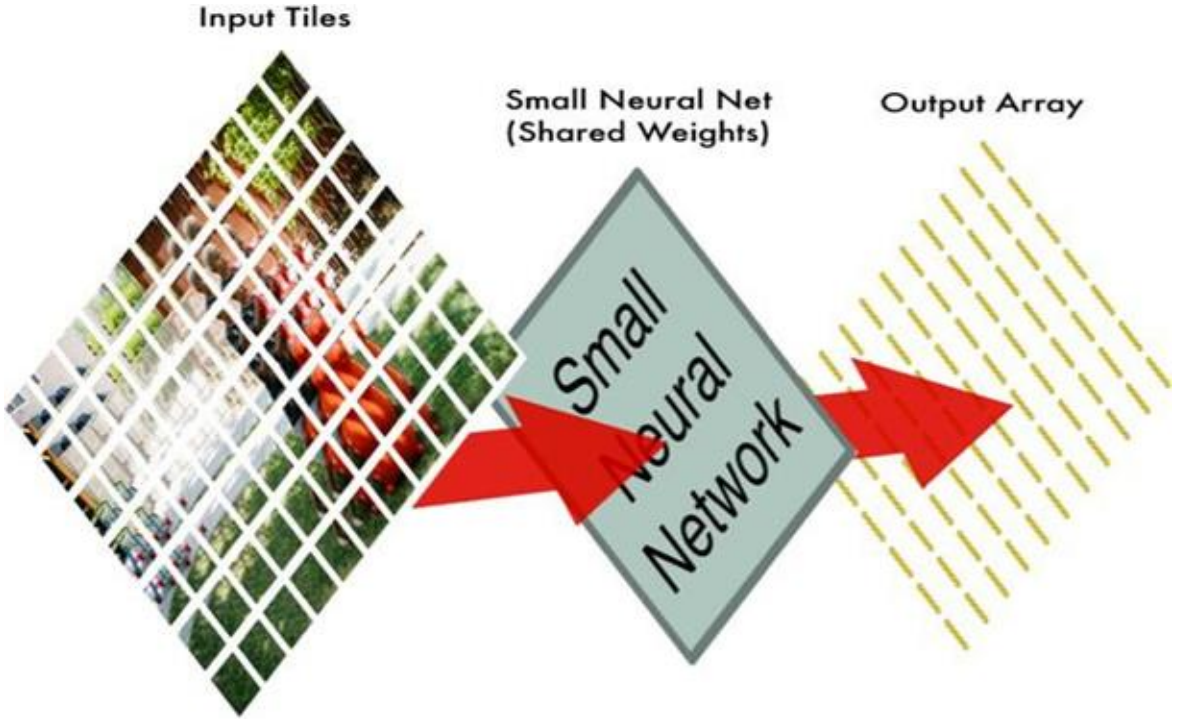
How Convolution Works? (cont.)

.2We process each tile using the weights matrix of a small neural network. We call this matrix a **kernel** or **filter**.



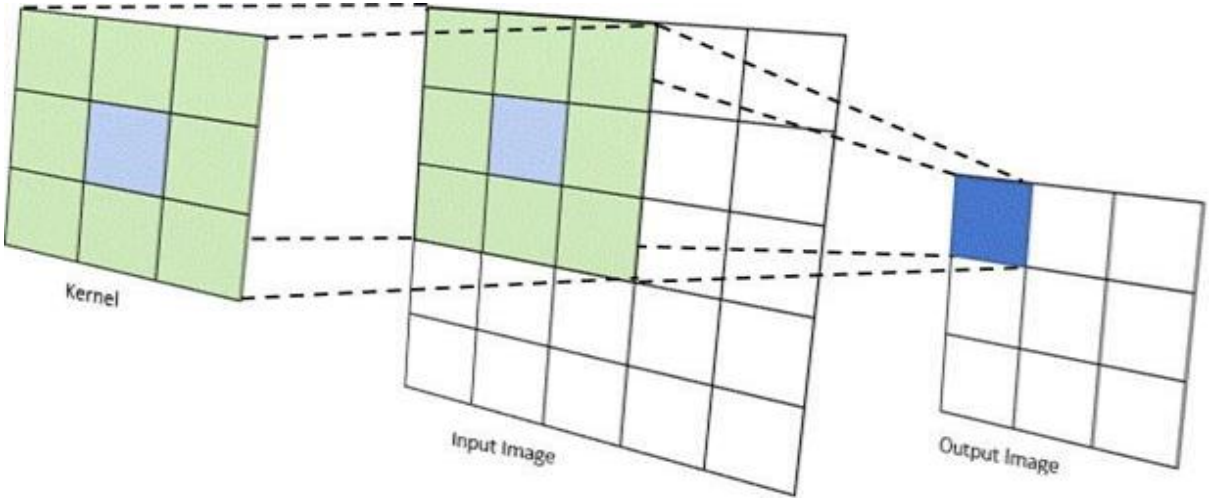
How Convolution Works? (cont.)

.3Finally, the filter slides across the image (with the same weights) and processes all the tiles, creating an output **feature map**.



How Convolution Works? (cont.)

- What we did in the last step is called the **Convolution Operation**.
- We convolved the kernel with the image, which means sliding the kernel over the entire image and computing the dot product between the kernel and small regions of the image at each step.



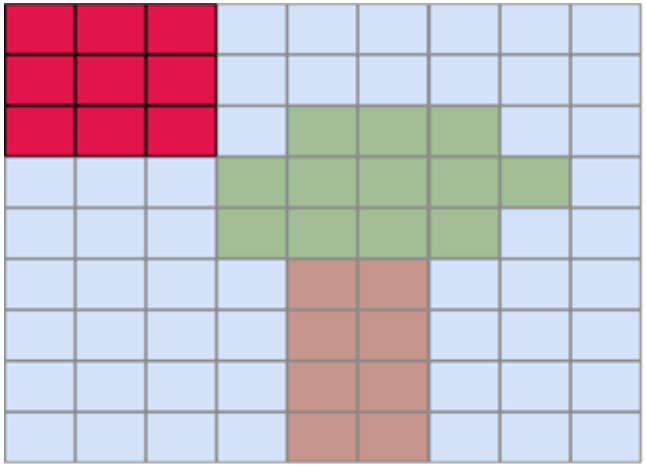
How Convolution Works? (cont.)

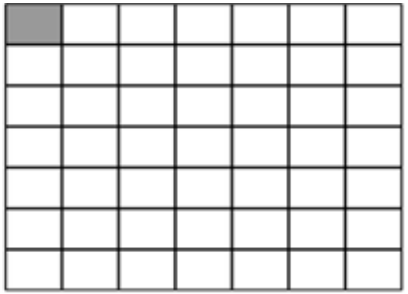
- This can be represented mathematically by:

$$z = W * x_{i,j} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} W_{a,b} X_{(i+a)(j+b)}$$

- z) ta noitulovnoc eht fo tuptuO :i, j.(
- W ezis fo xirtam (lenrek) retliF :$m \times n$.
- xi,j) noitisop ta eulav tupnI :i, j.(
- Wab) ta retlif eht fo thgieW :a, b.(
- $X)i+a)(j+b)$ ta noiger llams eht ni eulav tupnI :$(i+a, j+b$.(

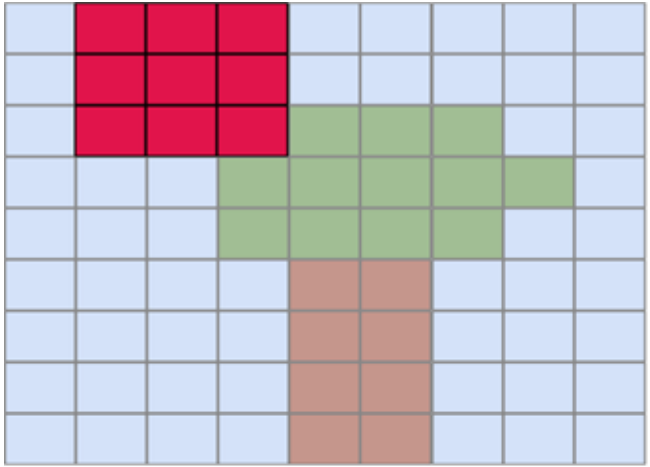
How Convolution Works? (cont.)

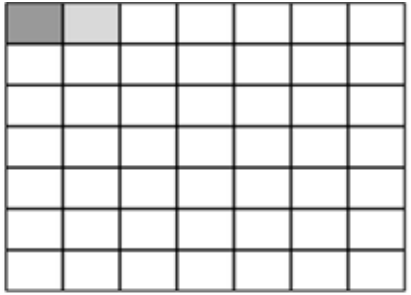




The **kernel** slides across the image and produces an output value at each position

How Convolution Works? (cont.)





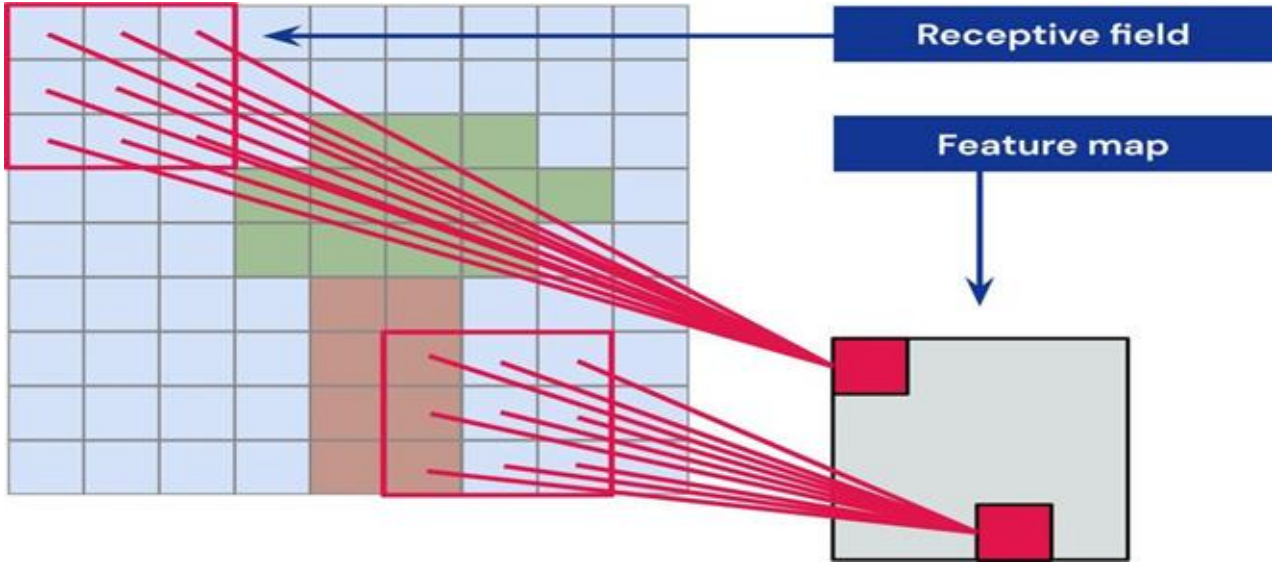
The **kernel** slides across the image and produces an output value at each position

How Convolution Works? (cont.)

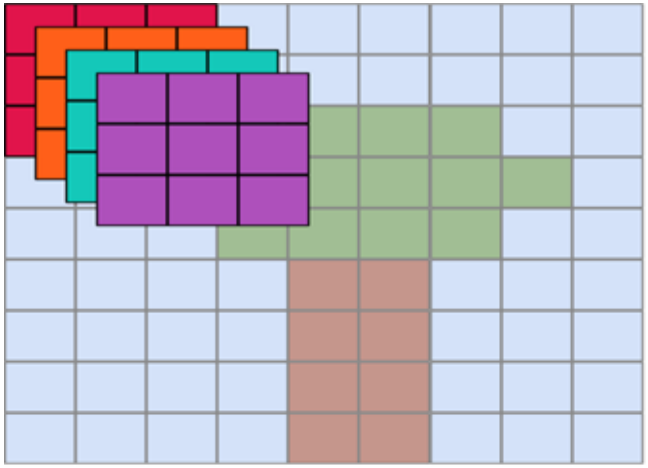
The **kernel** slides across the image and produces an output value at each position

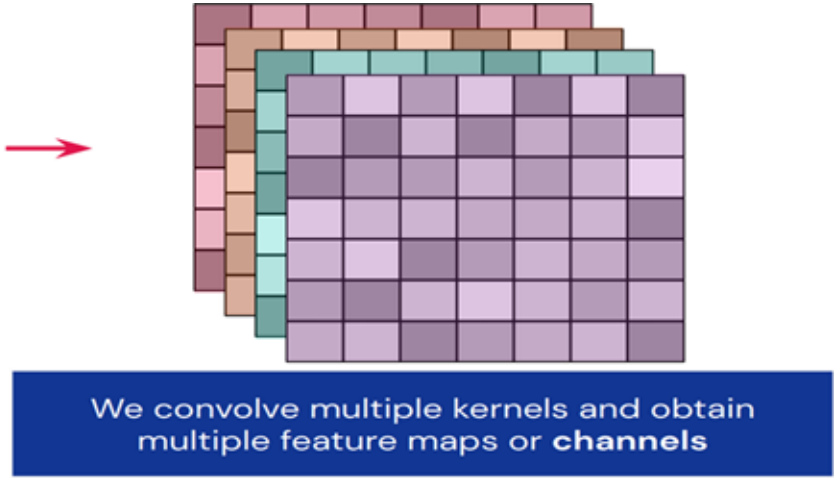
How Convolution Works? (cont.)

- **Receptive Field** lenrek eht taht egami tupni eht fo noiger ehT :
  operates on at each step.
- **Feature Map**.egami tupni eht morf detcartxe serutaef ehT :



How Convolution Works? (cont.)





How Convolution Works? (cont.)

- **Filters Example** ,segde ekil snrettap tceted sretlif tnereffiD : textures, or smoothness, producing corresponding feature maps.

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

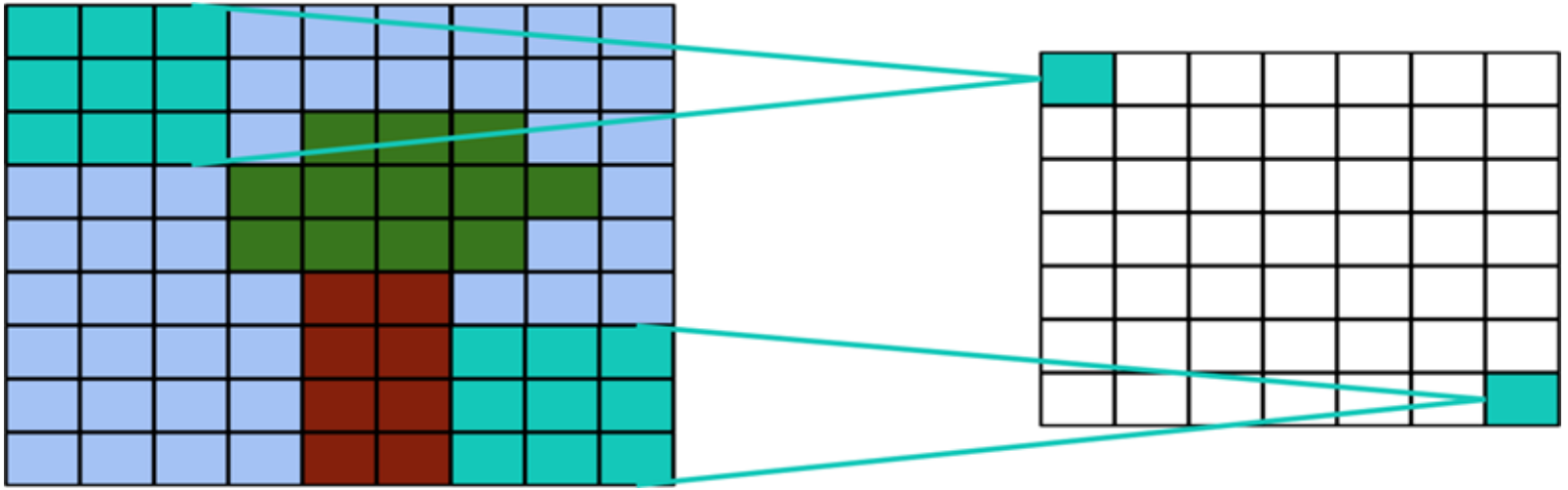


$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \qquad \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \qquad \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



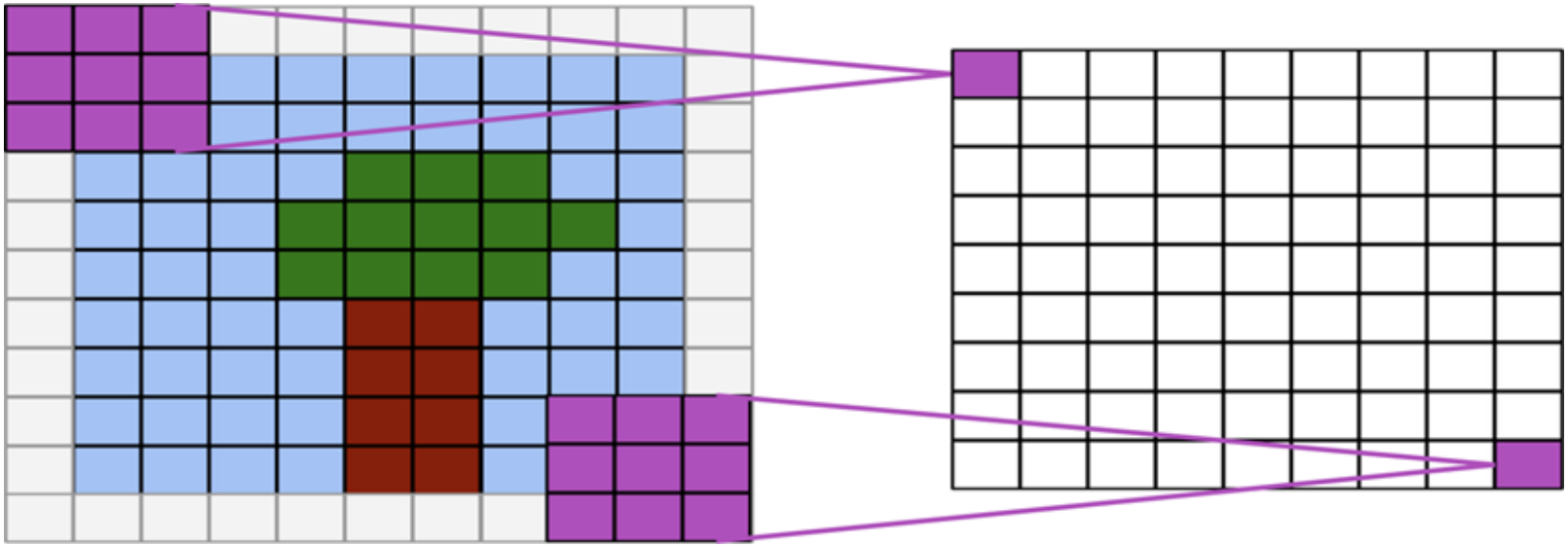
Controlling the Convolution Process

- Applying Convolution as such reduces the size of the borders.
- Sometimes this is not desirable.



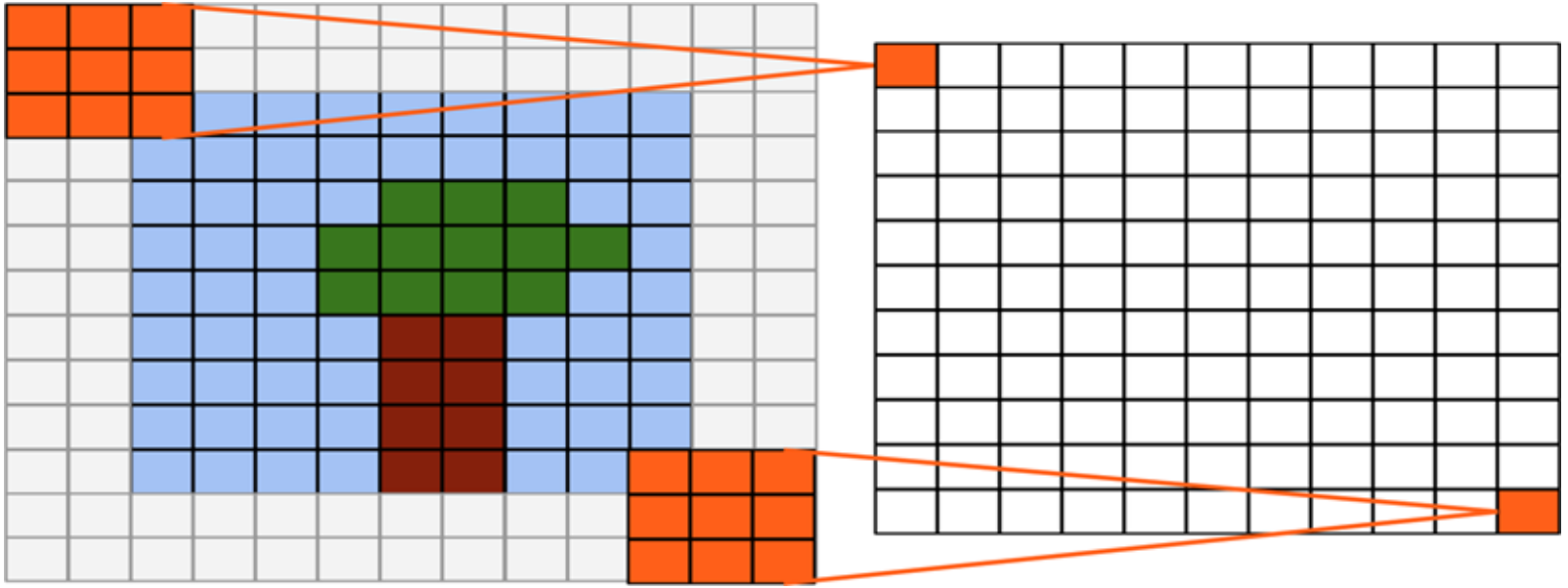
Controlling the Convolution Process (cont.)

- Solution: We can use **Padding**.sorez htiw redrob eht sdap hcihw ,
- it has two types:
  1. **Same Convolution** eht slauqe ezis tuptuo eht os dedda si gniddaP : .ezis tupni



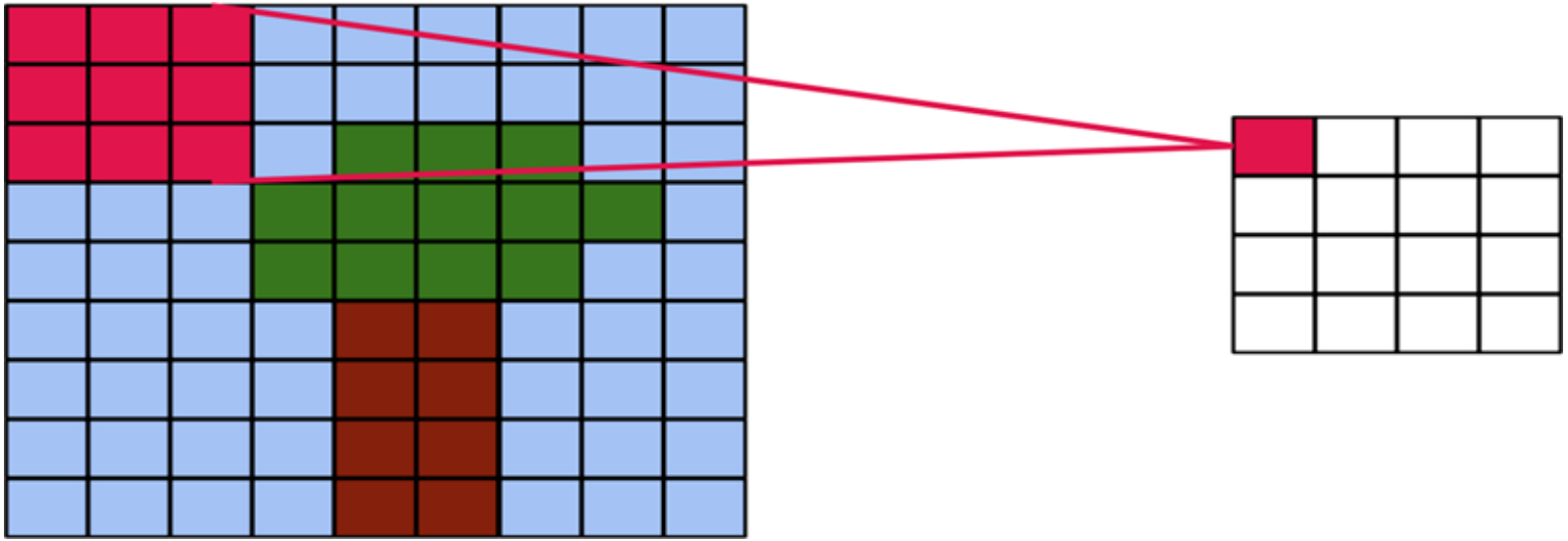
Controlling the Convolution Process (cont.)

- 1. **Full Convolution** eritne eht srevoc lenrek eht os dedda si gniddaP :
  .segde gnidulcni ,tupni
- output size = input size + kernel size - 1



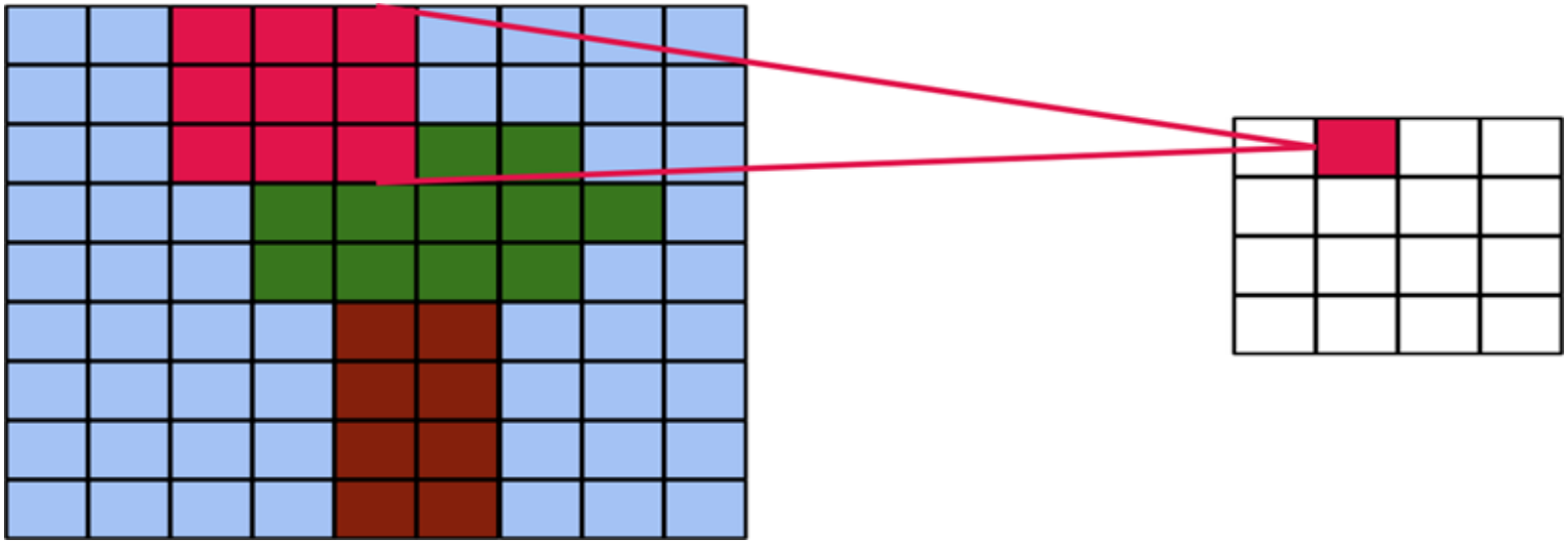
Controlling the Convolution Process (cont.)

- **Strided Convolution**1 < pets a htiw egami eht gnola sedils lenreK :
- Larger strides reduce output size.



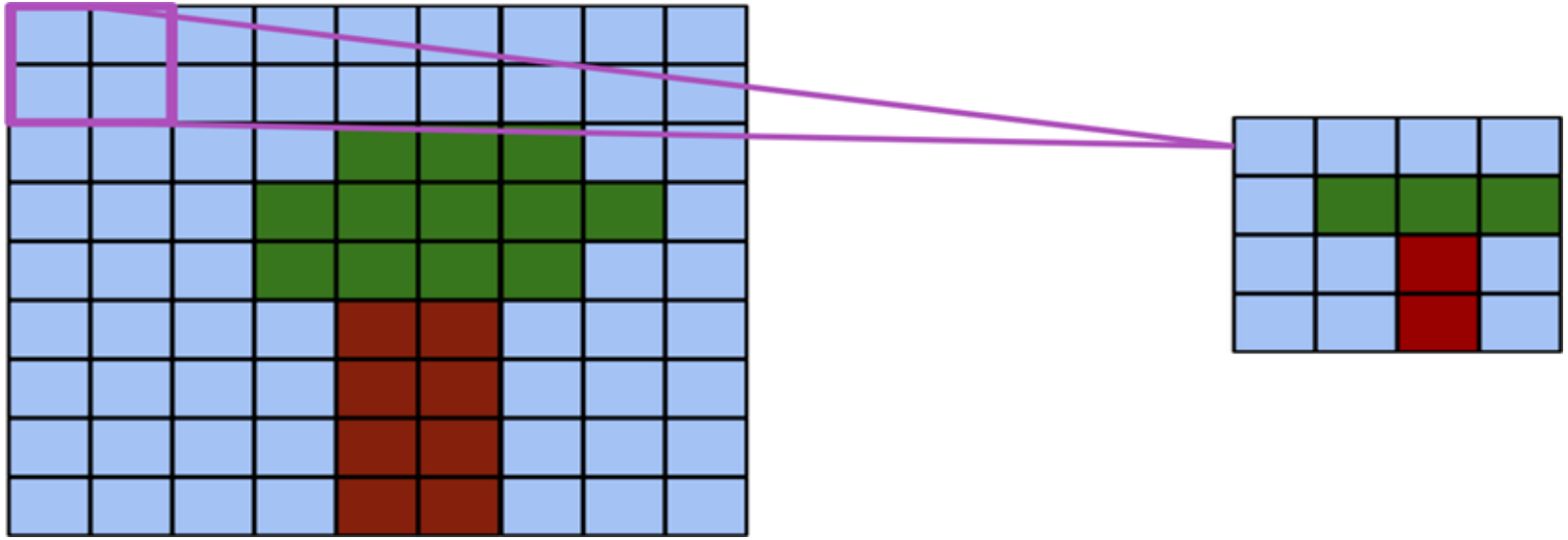
Controlling the Convolution Process (cont.)

- **Strided Convolution**1 < pets a htiw egami eht gnola sedils lenreK :
- Larger strides reduce output size.

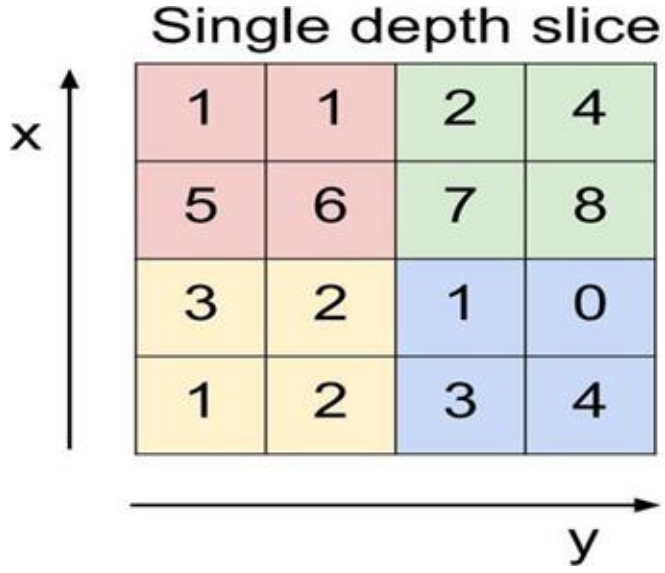


Controlling the Convolution Process (cont.)

- **Pooling** ecuder ot swodniw llams revo xam ro naem etupmoC : resolution and extract more general features.



Controlling the Convolution Process (cont.)



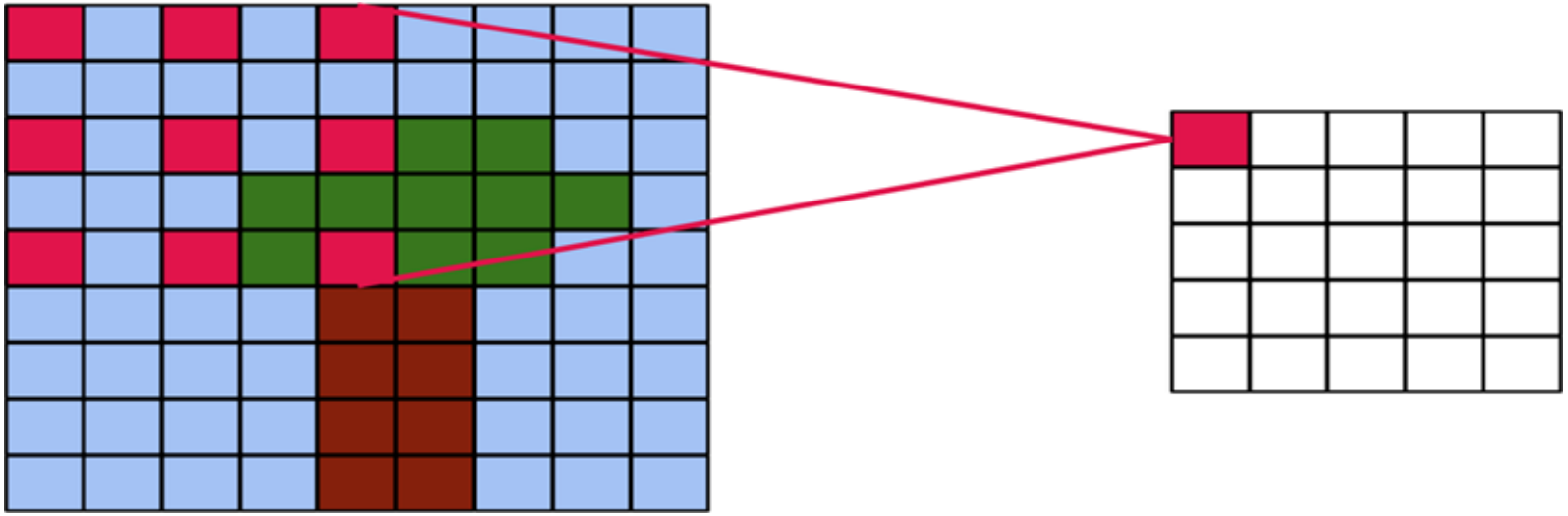
max pool with 2x2 filters
and stride 2

6	8
3	4

- No learnable parameters
- Introduces spatial invariance

Controlling the Convolution Process (cont.)

- **Dilated Convolution**1 < pets ,tuo daerps si lenreK :between kernel elements.
- It expands the receptive field without increasing the number of parameters, which makes it efficient.



CNN Output size

$$n_{out} = \frac{n_{in} 2 + p - k}{s} 1 +$$

=

nin serutaef tupni fo rebmun :

nout serutaef tuptuo fo rebmun :

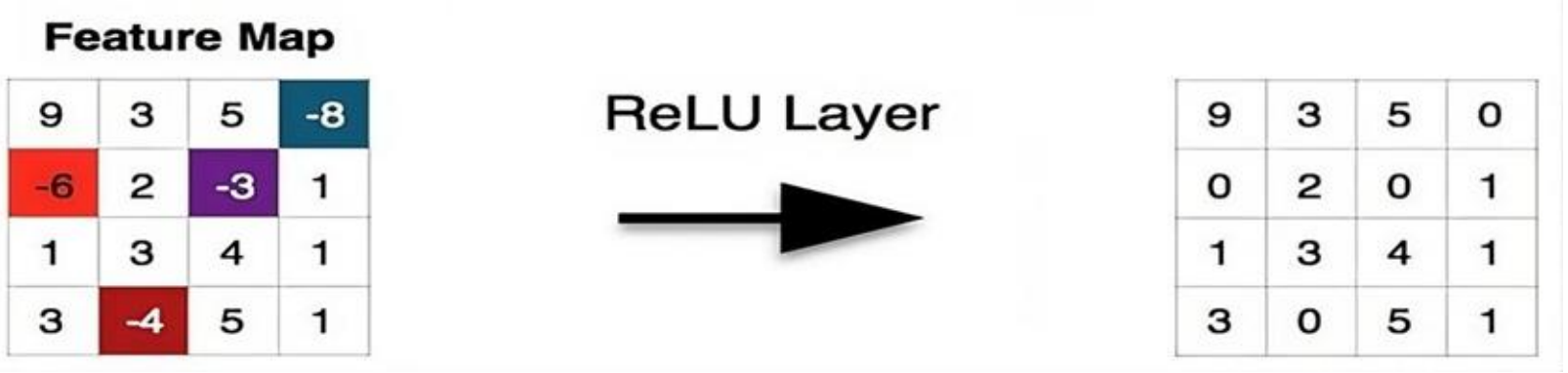
k ezis lenrek noitulovnoc :

p ezis gniddap noitulovnoc :

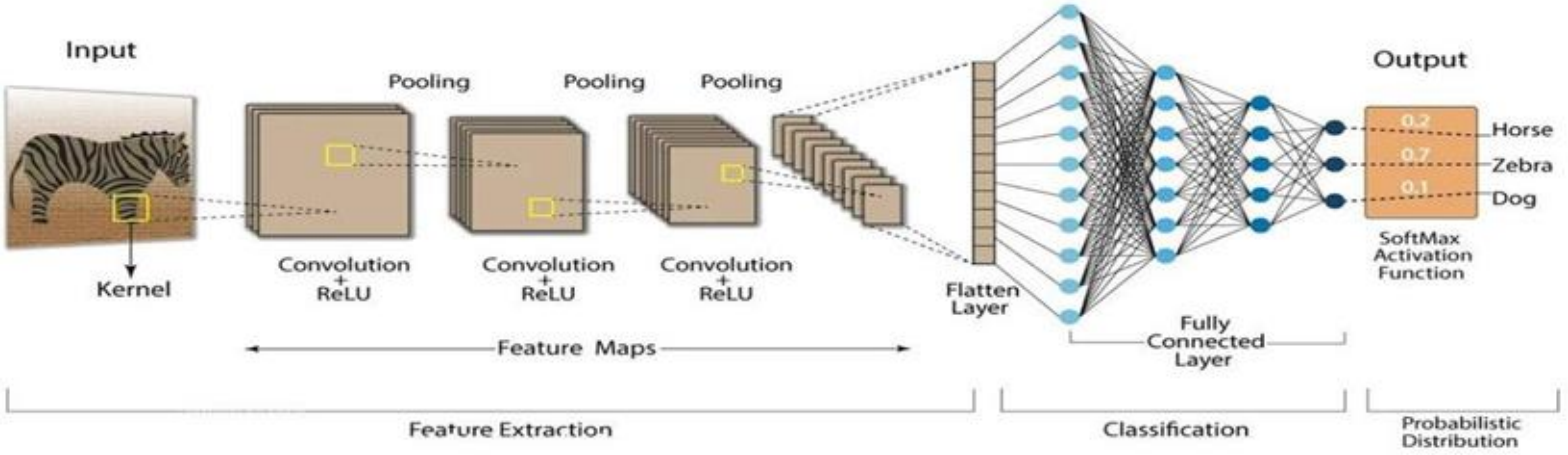
s ezis edirts noitulovnoc :

Activation

- Just like Fully-Connected Neural Networks, we can apply an activation over convolutional layer outputs
- It helps break linearity
- For example, Rectified Linear Unit (ReLU): $\sigma x = (max(x, 0)$

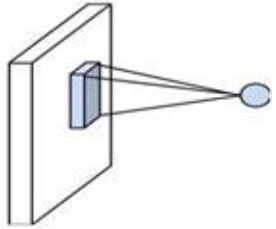


Convolutional Neural Networks

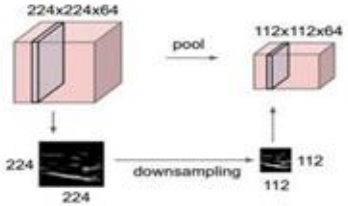


Components of a CNN

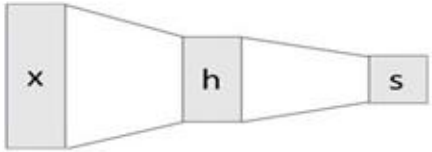
Convolution Layers



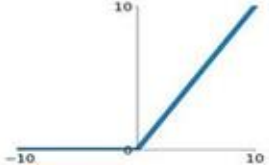
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

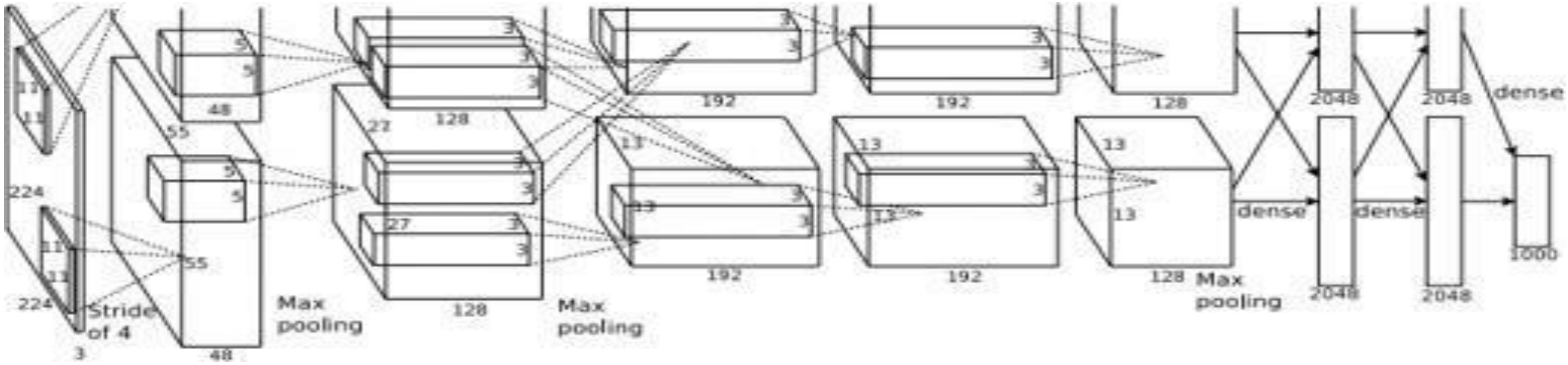
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Most Notable CNN-based Architectures

- Over time, researchers built advanced CNN architectures to improve performance and efficiency. These architectures introduced key innovations:
  - **AlexNet [Krizhevsky et al.** eveihca ot NNC tsrif ehT :**[2012** breakthrough performance on image classification.
  - **VGGNet [Simonyan and Zisserman,** peed yrev desU :**[2014** networks (up to 19 layers).
  - **InceptionNet (GoogLeNet) [Szegedy et al.,** elpitlum desU :**[2014** filter sizes per layer (Inception modules).
  - **ResNet [He et al.,** gniniart rof snoitcennoc piks decudortnI :**[2015** very deep networks.
  - **EfficientNet [Tan and Le,** taht dohtem gnilacs a dnuoF :**[2019** simultaneously scales a CNN's depth, width, and resolution optimally using a single scaling coefficient.

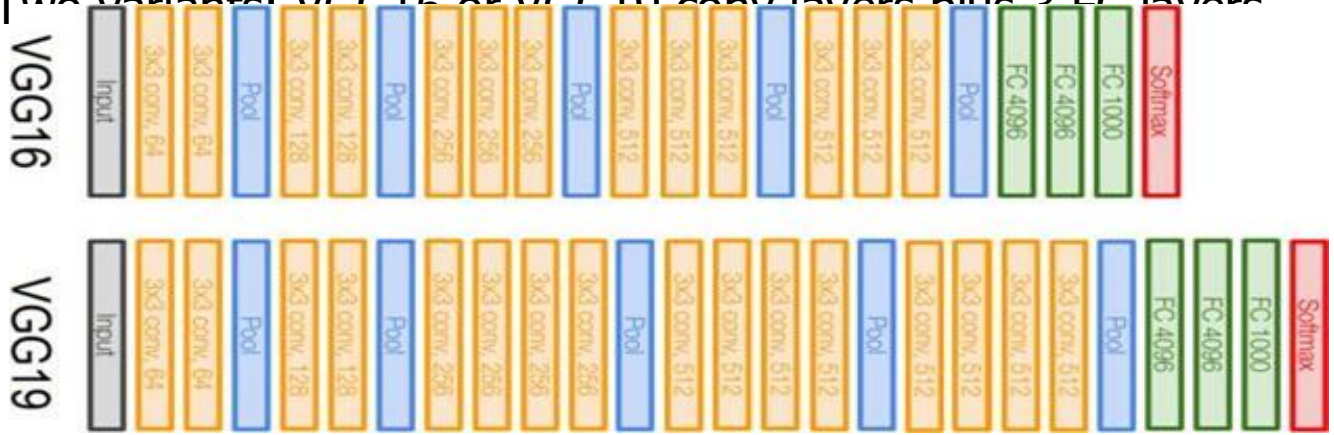
AlexNet

- First big improvement in image classification.
- Made use of CNN, pooling, dropout, ReLU and training on GPUs.
- 5convolutional layers, followed by max-pooling layers; with three fully connected layers at the end



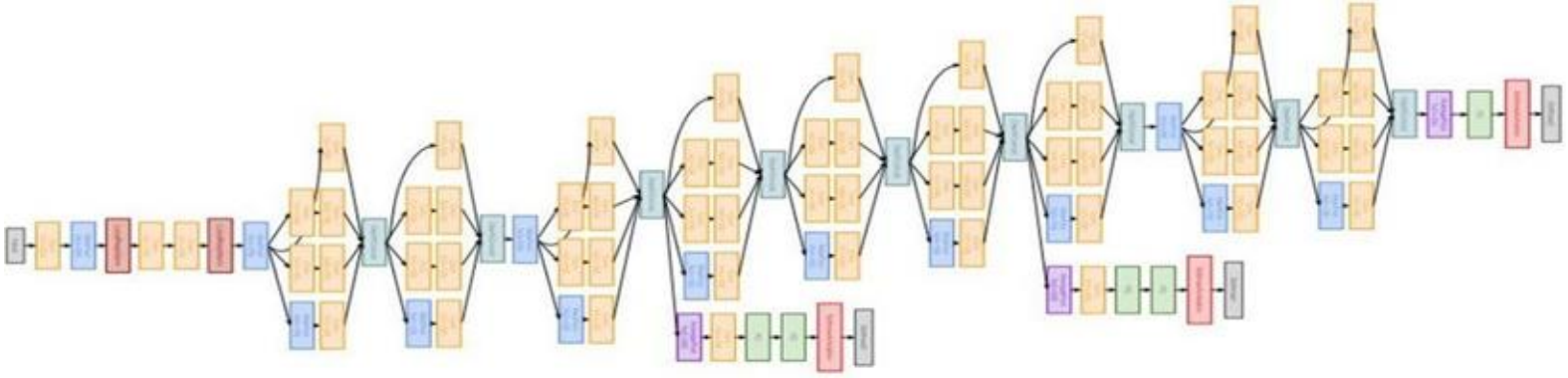
VGGNet

- **Improvement over AlexNet:** Uses a deeper network with small filters instead of a shallow network with larger filters.
- A stack of 3x3 conv layers (vs. 11x11 conv) has same receptive field, more non-linearities, fewer parameters and deeper network representation.
- Two variants: VGG16 or VGG19 conv layers plus 3 FC layers



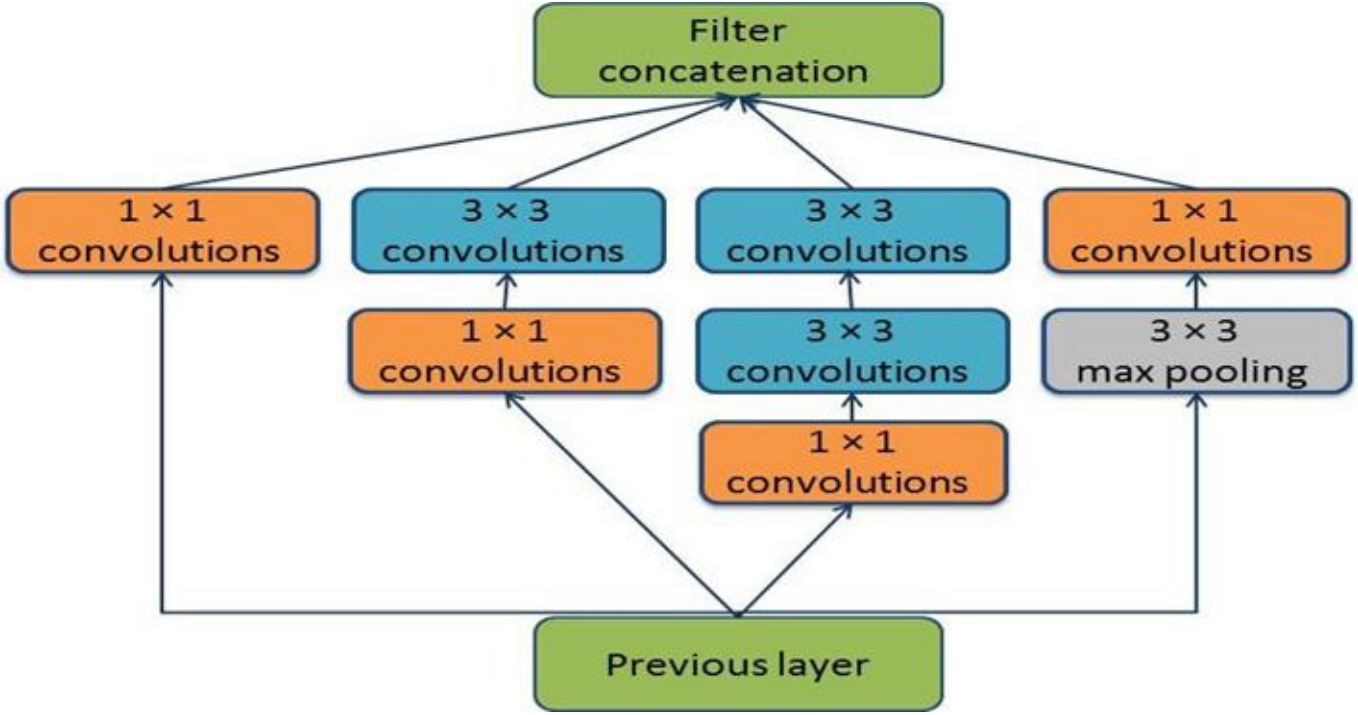
InceptionNet

- Going Deep: 22 layers
- Only 5 million parameters! (rewef ×27 ,teNxelA naht rewef ×12 than VGGNet).
- Introduced efficient ”Inception module”
- Introduced ”bottleneck ”layers that use 1x1 convolutions to reduce the number of channels and mix information across them.



InceptionNet (cont.)

- **Inception module:** Uses multiple filter sizes (ni ,(5×5 ,3×3 ,1×1 parallel, to capture different features, then combines their

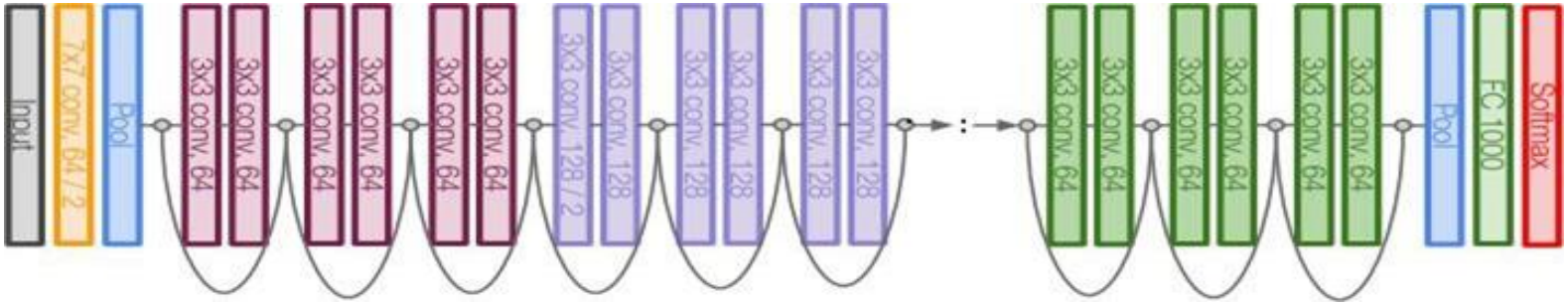


ResNet: Motivation

- **Problem:** Making networks deeper does not always improve accuracy.
- **Why?** In very deep networks, gradients become extremely small as they move backward through layers, making learning slow or stopping it altogether (**vanishing gradient problem**.(
- **Solution:** Residual Network (ResNet) introduces **skip connections** .ylisae erom wolf ot noitamrofni gniwolla ,**(slaudiser)**

ResNet

- Very deep networks using residual connections
- -152layer model for ImageNet
- Stacked Residual Blocks
- **Residual:** A shortcut connection that helps the network pass information through layers more easily.

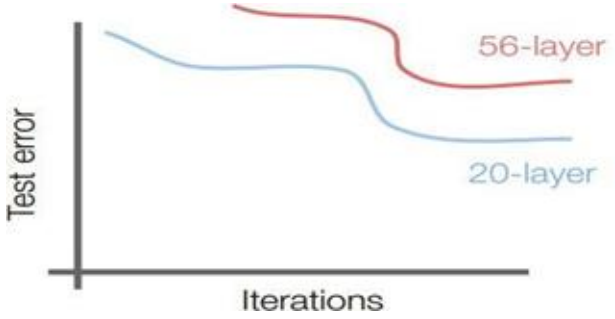


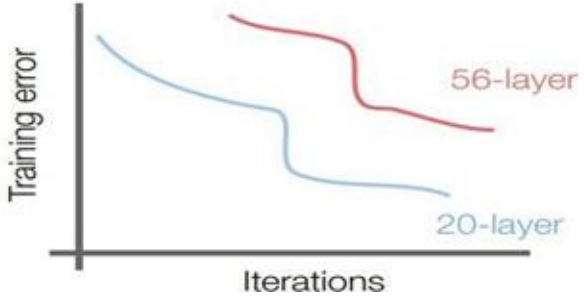
ResNet

- What happens when we continue stacking deeper layers on a ”plain ” convolutional neural network?

ResNet

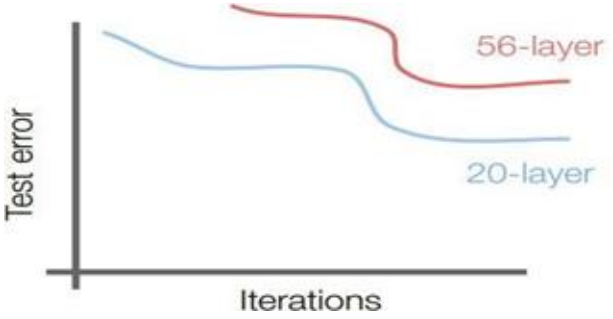
- What happens when we continue stacking deeper layers on a ”plain ” convolutional neural network?

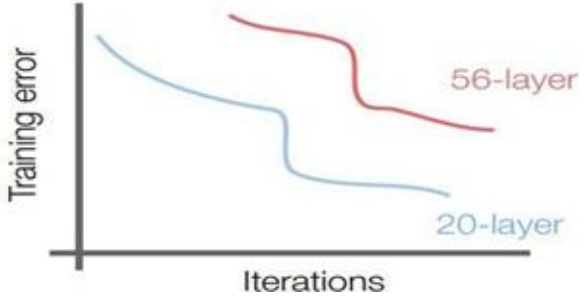




ResNet

- What happens when we continue stacking deeper layers on a ”plain ” convolutional neural network?

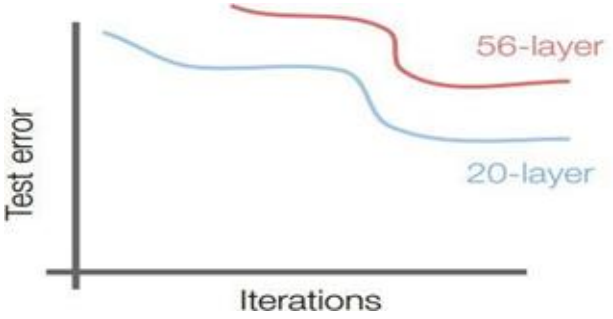


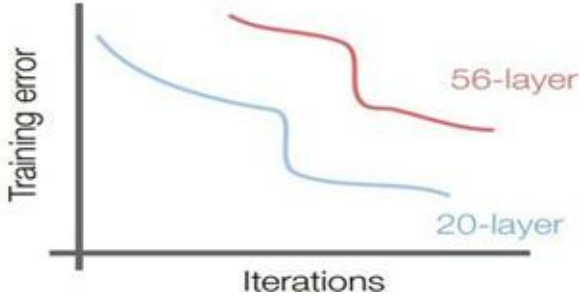


- -56layer model performs worse on both test and training error

ResNet

- What happens when we continue stacking deeper layers on a ”plain ” convolutional neural network?





- -56layer model performs worse on both test and training error
- The deeper model performs worse, but it’s not caused by overfitting!

ResNet

- **Fact:** Deep models have more representation power (more parameters) than shallower models.

ResNet

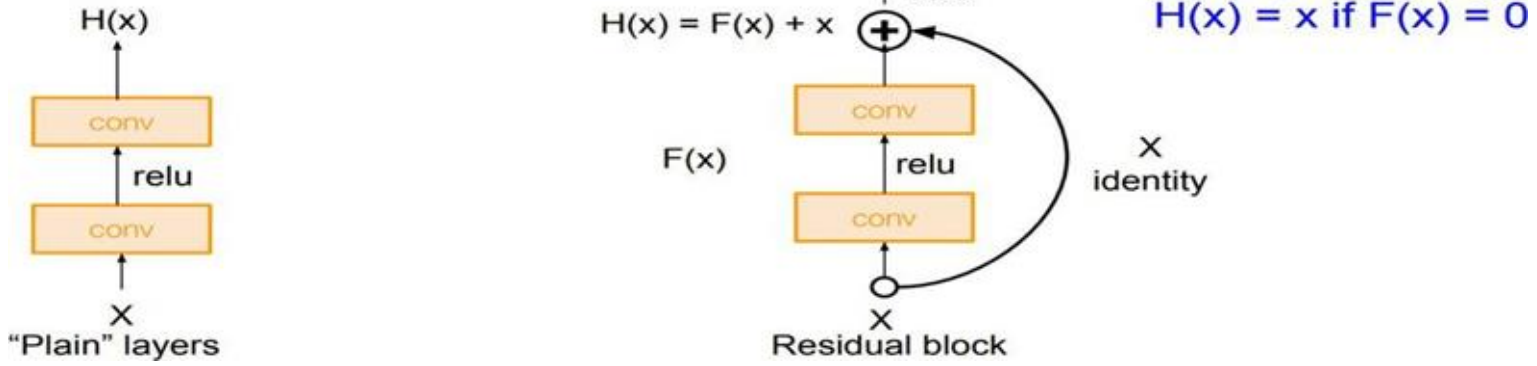
- **Fact:** Deep models have more representation power (more parameters) than shallower models.
- **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize

ResNet

- **Fact:** Deep models have more representation power (more parameters) than shallower models.
- **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize
- **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

ResNet

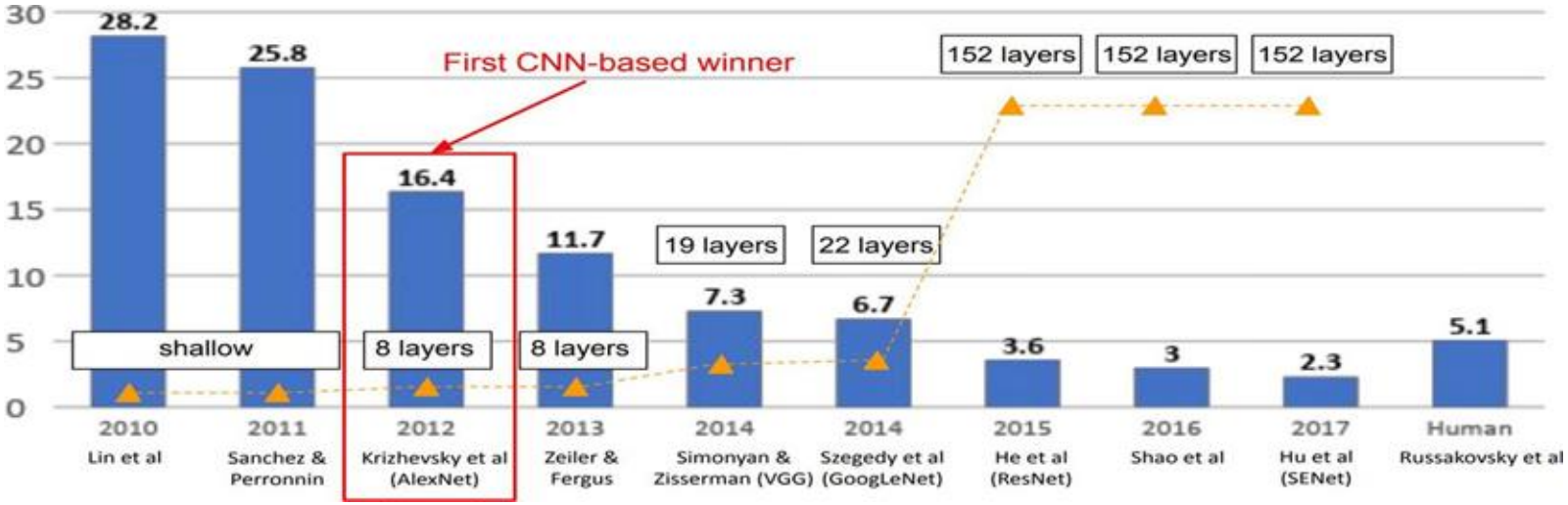
- **Fact:** Deep models have more representation power (more parameters) than shallower models.
- **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize
- **Solution:** Use network layers to fit a residual mapping instead of



ImageNet

- The most extensive data for Image Classification
- 3RGB channels from 0 to 255
- 14,197,122images
- 1000classes

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



EfficientNet: Motivation

- **Problem:** To improve accuracy, we can:
  - Increase the number of layers (**depth**(
  - Increase the number of neurons in each layer (**width**(
  - Use higher-resolution images (**resolution**(

  But finding the right balance of these three was largely based on trial and error.
- **Solution:** EfficientNet introduced a **compound scaling** method—a mathematical formula to systematically find the optimal balance among **depth** ,**width** dna ,**resolution**.

EfficientNet

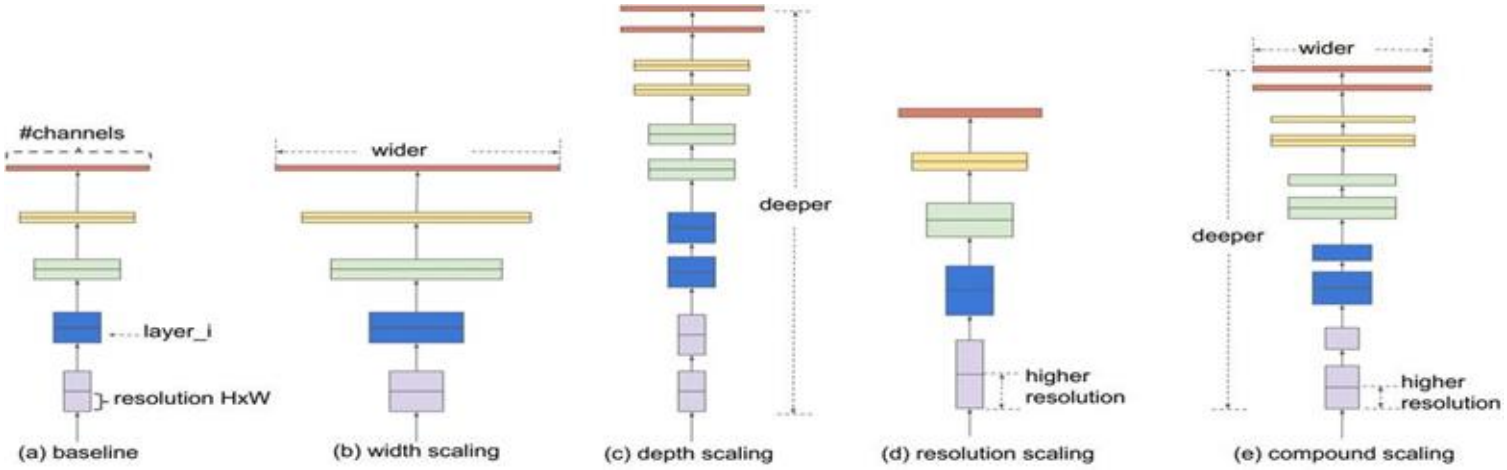


Figure .gnilacS teNtneiciffE :5

EfficientNet

- **Compound Scaling** Uses a single **scaling coefficient** ot (ϕ) control:
  - **Network Depth** → ($\phi\alpha$)More layers
  - **Network Width** → ($\phi\beta$)More channels per layer
  - **Input Resolution** → ($\phi\gamma$)Larger input images
- The goal: find γ, β, α that balance accuracy & efficiency, then scale up optimally by increasing the global coefficient .ϕ

EfficientNet

- EfficientNet optimizes depth, width, and resolution using this constraint:

$$2 \approx^2 \gamma \cdot^2 \beta \cdot \alpha$$

- **Why this equation?**
  - Increasing **depth** sPOLF sesaercni (α)**linearly**.
  - Increasing **width** sPOLF sesaercni (β)**quadratically** .($^2\beta$)
  - Increasing **resolution** sPOLF sesaercni (γ)**quadratically** .($^2\gamma$)
- To double total FLOPs, the three factors must be balanced together.

EfficientNet

- The authors of EfficientNet searched for the best scaling factors on a small baseline model.
- They found:

$\alpha = 1.2$, $\beta = 1.1$, $\gamma = 1.15$

EfficientNet Scaling: B0 to B7

- The EfficientNet family (B0 to B:gnisu detareneg si (7

Depth = α^{ϕ} 1.2 Width = β^{ϕ} 1.1 Resolution = γ^{ϕ} 1.15

Model	ϕ	Depth ($\phi\alpha$)	Width ($\phi\beta$)
B0	0	$1.0 = 1.2^0$	$1.0 = 1.1^0$
B1	1	$1.2 = 1.2^1$	$1.1 = 1.1^1$
B2	2	$1.44 = 1.2^2$	$1.21 = 1.1^2$
B3	3	$1.73 = 1.2^3$	$1.33 = 1.1^3$
B4	4	$2.07 = 1.2^4$	$1.46 = 1.1^4$
B5	5	$2.49 = 1.2^5$	$1.61 = 1.1^5$
B6	6	$2.99 = 1.2^6$	$1.77 = 1.1^6$
B7	7	$3.58 = 1.2^7$	$1.94 = 1.1^7$

Table 0 B morf teNtneiciffE gnilacS :1to B7

- We multiply these scaling factors by the baseline EfficientNet-B0 values and round to the nearest integer to get the new depth, width, and resolution for each model.

EfficientNet

- EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.

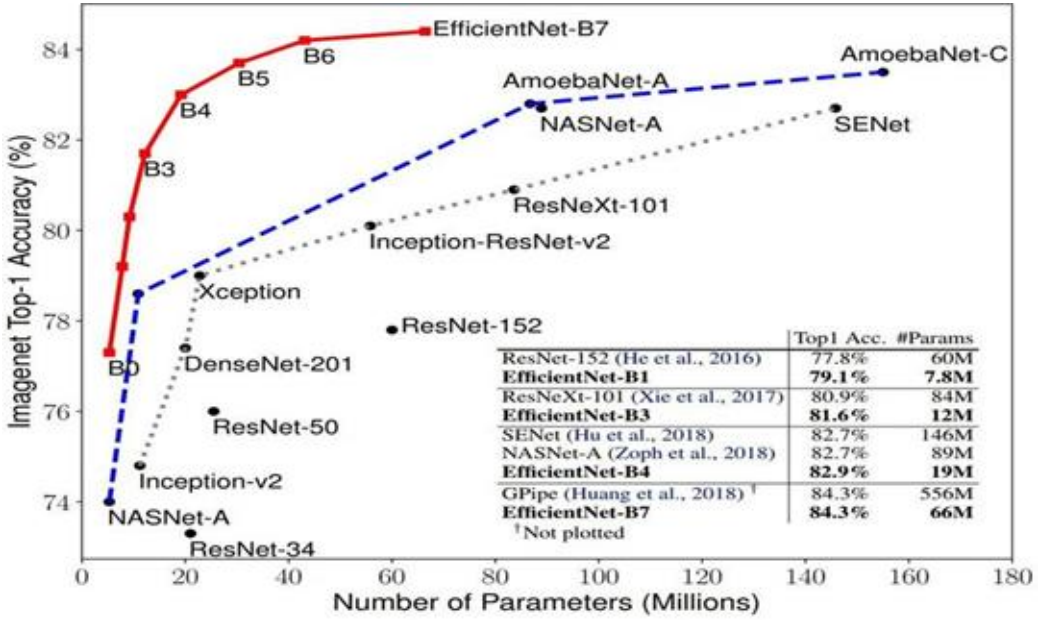
EfficientNet

- EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.
- EfficientNet-B7 reaches 84.4% Top-1 and 97.3% Top-5 accuracy on ImageNet.

EfficientNet

- EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.
- EfficientNet-B7 reaches 84.4% Top-1 and 97.3% Top-5 accuracy on ImageNet.
- More efficient than previous CNN models8.4—x smaller and 6.1x faster than competitors.

EfficientNet Performance



	Top1 Acc.	#Params
ResNet-152 (He et al., 2016)	77.8%	60M
EfficientNet-B1	**79.1%**	**7.8M**
ResNeXt-101 (Xie et al., 2017)	80.9%	84M
EfficientNet-B3	**81.6%**	**12M**
SENet (Hu et al., 2018)	82.7%	146M
NASNet-A (Zoph et al., 2018)	82.7%	89M
EfficientNet-B4	**82.9%**	**19M**
GPipe (Huang et al., 2018) †	84.3%	556M
EfficientNet-B7	**84.3%**	**66M**

† Not plotted

Figure .tenegamI no ecnamrofreP teNtneiciffE :6

Motivation for MobileNets

- **Why MobileNets?**
  - Small-sized models are crucial for mobile and embedded devices.
  - MobileNets reduce computational cost and memory usage while maintaining good accuracy.
- **Key Idea:**
  - Use **depthwise-separable convolutions** to significantly reduce computation compared to standard convolutions.

Computational Cost of Convolutions

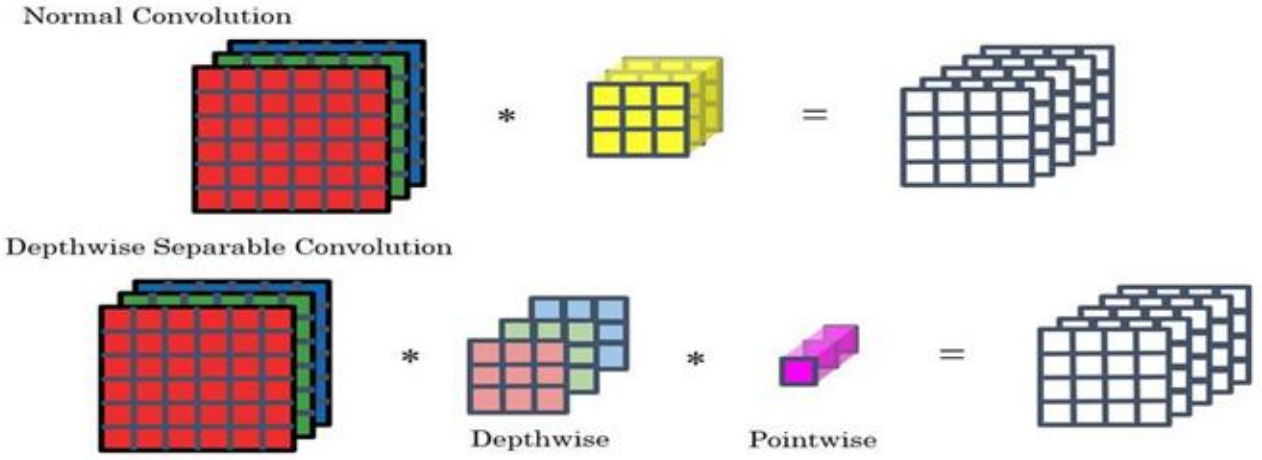
- **Computational cost of standard convolution:**

Cost = # filter params × # filter positions × # filters

- Filters operate on all input channels, increasing computation significantly.

Depthwise-Separable Convolutions

- Split standard convolution into two steps:
  - **Depthwise Convolution:** Applies a single filter per input channel.
  - **Pointwise Convolution:** Combines outputs from depthwise convolution.
- **Key Benefit:** Reduces computational cost significantly compared to standard convolution.



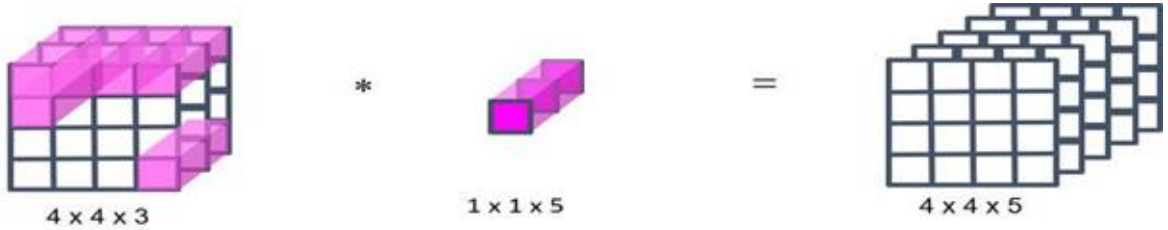
Depthwise Convolution

- Operates on each input channel separately.

Computational cost = #filter params x # filter positions x # of filters

Pointwise Convolution

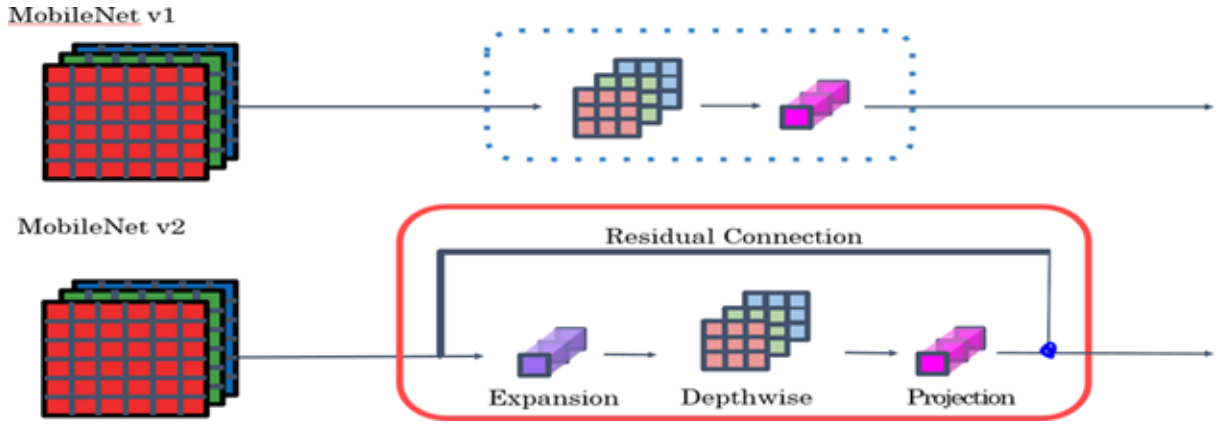
- Combines outputs from depthwise convolution using 1 ×1 convolutions (mixes channels).



Computational cost = #filter params x # filter positions x # of filters

MobileNet Architectures

- **MobileNet v:2**
  - Adds **residual connections**.
  - Introduces:
    - **Expansion step:** Expands input dimensions before depthwise convolution.
    - **Projection step:** Reduces dimensions after processing.



Pipeline Corrosion & Defect Analysis

- **CNN Architectures Used:**
  - ResNet
  - VGG
  - InceptionNet
- **Use Case:**
  - Automated analysis of pipeline conditions using camera feeds.
  - Classification of corrosion severity levels.
  - Predictive maintenance through visual pattern recognition.

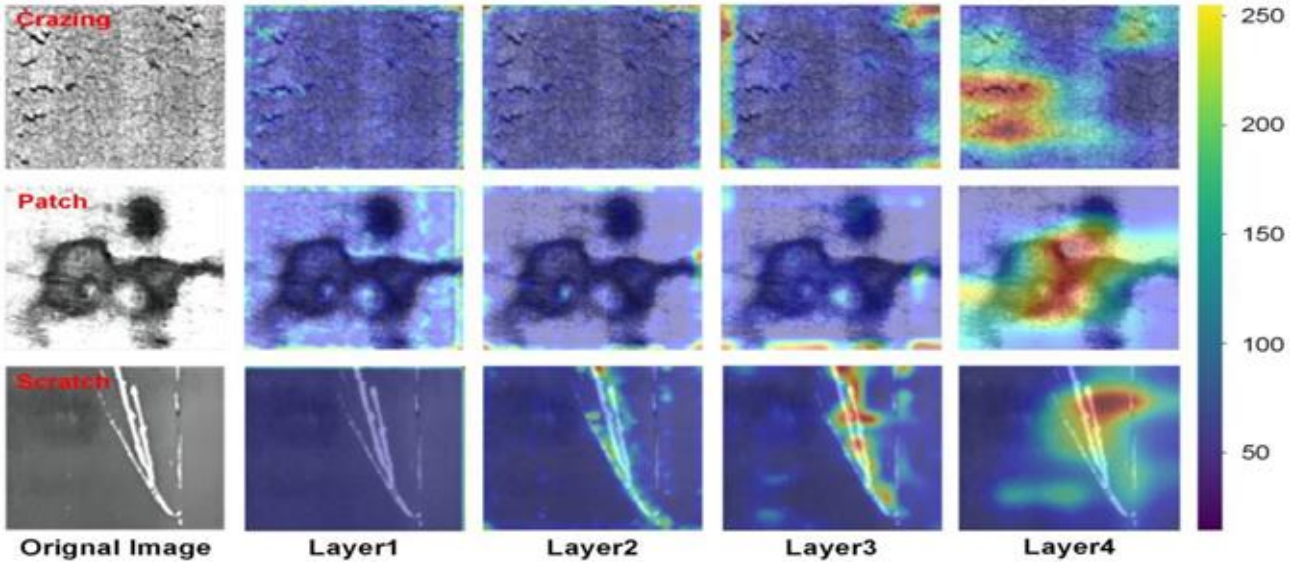
Pipeline Corrosion & Defect Analysis (cont.)

- **Benefits:**
  - Reduces manual inspection costs and time.
  - Early detection prevents catastrophic failures.
  - Enables data-driven maintenance scheduling.
- **Links & Resources:**
  - viso.ai
  - ScienceDirect
  - NextBrain

Equipment Defect Classification

- **CNN Architectures Used:**
  - ResNet50 with Transfer Learning
  - VGG19
  - AlexNet
- **Use Case:**
  - Classify types of defects in steel surfaces and equipment.
  - Automated quality control for manufactured components.
  - Multi-class defect recognition (rust, cracks, deformation, etc.).

Equipment Defect Classification (cont.)



- **Benefits:**
  - %99.4accuracy with transfer learning approaches.
  - Cost savings through early detection.
  - Replaces subjective manual inspections.

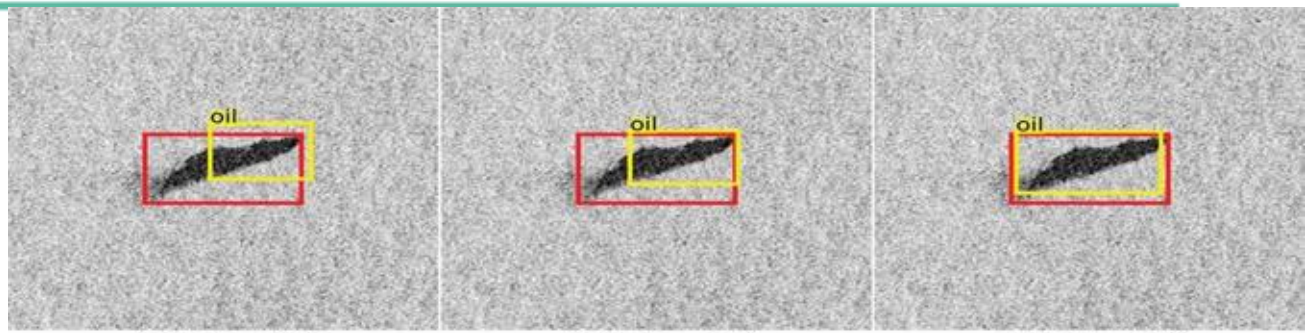
Equipment Defect Classification (cont.)

- **Links & Resources:**
  - MDPI: Steel defect detection (ResNet, 99.4% accuracy)

Oil Spill Detection & Classification

- **CNN Architectures Used:**
  - YOLOv4
  - Mask R-CNN
  - CNNs
- **Use Case:**
  - Classify oil spill severity and type from SAR/optical imagery.
  - Automated environmental monitoring systems.
  - Multi-class classification for response prioritization.

Oil Spill Detection & Classification (cont.)



(a) IoU = 30% (b) IoU = 45% (c) IoU = 75%

- **Benefits:**
  - Faster emergency response decisions.
  - Accurate spill characterization for cleanup planning.
  - Compliance with environmental regulations.
- **Research Papers & Resources:**
  - Deep learning oil spill detector (Sentinel-1 SAR, (2022
  - Comprehensive review of oil spill DL methods(2024)

References

These slides have been adapted from

- Fei-Fei Li, Yunzhu Li & Ruohan Gao, Stanford CS231n: Deep Learning for Computer Vision
- Assaf Shocher, Shai Bagon, Meirav Galun & Tali Dekel, WAIC DL4CV Deep Learning for Computer Vision: Fundamentals and Applications
- Justin Johnson, UMich EECS rof gninraeL peeD :498.008/598.008 Computer Vision
- Sander Dieleman, Deepmind: Deep Learning Lecture Series 2020